Stock Gossip Monitor

Introduction

Purpose

What is the problem or the opportunity that the project is investigating?

The project is designed for fundamental listed equity investors, including myself.

It addresses a specific gap: the inability to conduct meaningful financial research while simultaneously engaging with peers or operating in a mobile setting. In practice, idea generation (conversations, “gossip”) and deep research are disconnected activities.

Traditional platforms such as Bloomberg, FactSet, and S&P Capital IQ provide vast amounts of information through desktop-oriented interfaces. While comprehensive, they tend to overwhelm users with breadth rather than deliver focused insight.

Furthermore, these tools are not well suited to mobile use, which reinforces a siloed workflow: idea generation happens in conversations, while deep analysis requires a desktop or laptop environment – typically in the office, hotel, or on a flight.

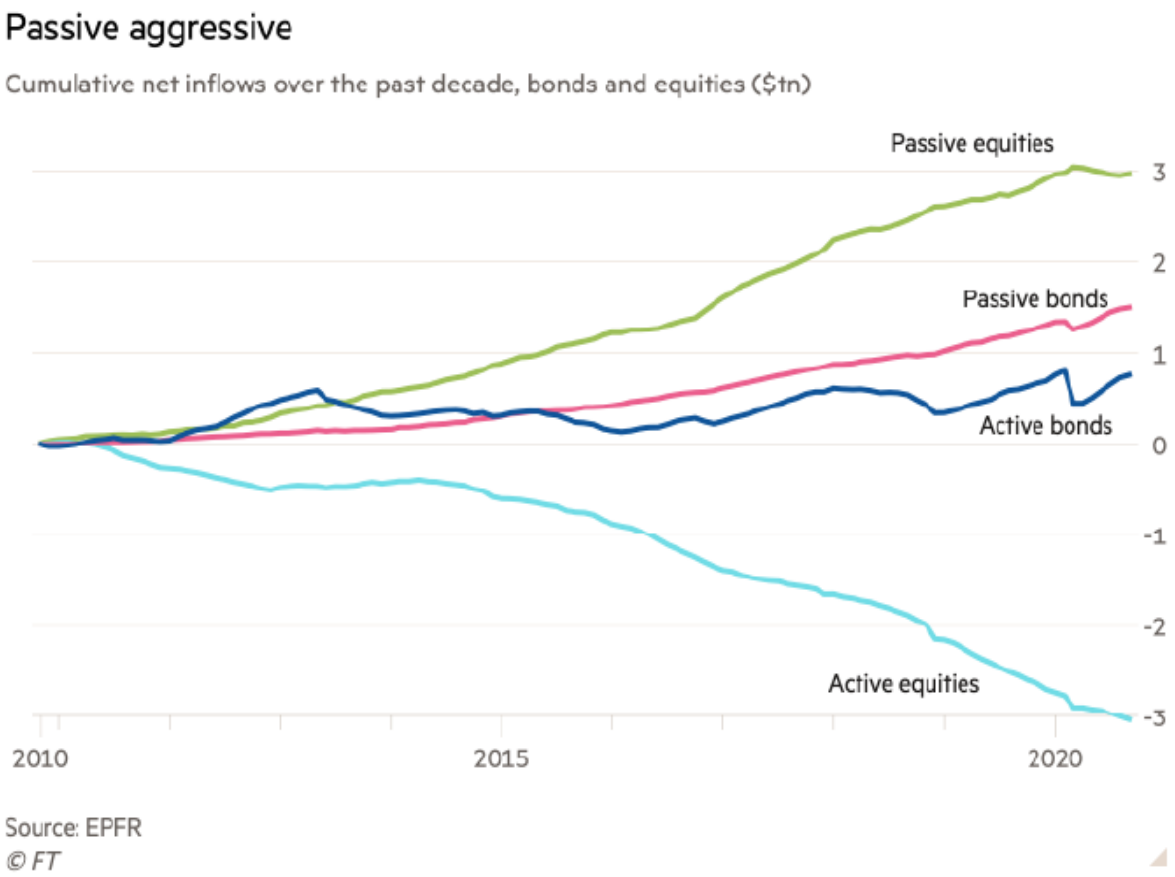
Why is this problem valuable to address?

In the extremely competitive world of listed markets with a preponderance of competition from leveraged pods, index funds and quants, fundamental investment professionals need faster relevant insights to compete.

Modern fundamental investment professionals understand that the predominant drivers of performance of an individual stock are the overall stockmarket, industry sector and certain style factors. Firm-specific news is only a small part of what drives share performance.

This is why there is a need for a software that, without being overly quantitative, quickly disaggregates thematic drivers of a share price from firm specific drivers. This is the thinking behind assigning investment categories to each stock, and designing the homepage as a list of Sector Cards.

The image below shows that in the last 15 years, traditional active investors have experienced US$3 trillion in outflows, while quantitative strategies (including passive) have gained US$3 trillion. This is why the problem is valuable.



What is the current state (e.g. unsatisfied users, lost revenue)?

Fundamental investment professionals are under pressure from outdated research processes and rigid style or marketing constraints.
These ways of working can make it harder to respond quickly to changing market conditions or to compete effectively with newer approaches such as quantitative strategies, index investing, and highly structured pod-manager models.

This can lead to several problems:

- too much time spent on low-value detail that does not meaningfully improve investment performance
- slower recognition of major market or factor shifts
- investment insights that are not sufficiently differentiated
- reduced competitiveness when trying to retain or win investment mandates

What is the desired state?

While a single app cannot solve the industry's larger structural challenges, it can address some of the symptoms.

The intended end state is to enable individual investors and small teams to:

i) respond decisively to thesis-changing developments while quickly filtering out irrelevant noise;

ii) size positions with greater conviction in those rare instances where company-specific, sector, and style factors align;

iii) engage in more informed and productive discussions with industry experts, fellow investors, and company management and boards;

iv) maintain a deeper pipeline of "next best" investment ideas for future portfolio inclusion;

v) operate with greater clarity, confidence, and reduced stress when navigating both tactical and strategic shifts in markets.

Problem & Solution

HYPOTHESIS

The Problem	The Solution	
Incumbent software: **too broad, too much**	WHO	**1–5 person boutique teams, family offices, self-directed HNW investors**
Investors **need to focus on metrics tailored to each investment category**	WHAT	**Harness for category-specific metrics** with **user overrides**
Deep research takes time and effort on Excel – **Excel doesn't scale well**	END VISION	Multi-API community research tool drawing from variety of data sources, **accessible anywhere**, AI-agent ready

Free up time for traditional equity investors to think more clearly and make more confident portfolio decisions

Has this problem been addressed by other projects? What were the outcomes?

There are adjacent solutions in the market – ranging from traditional research platforms to newer AI-enabled tools – that attempt to streamline investment workflows. However, most of these products either remain desktop-centric, overly broad, or insufficiently tailored to the specific, real-time needs of fundamental investors.

AI-driven tools have improved summarisation and data aggregation, but they tend to operate as overlays rather than deeply integrated, personalised research systems. As a result, the core problem – seamlessly combining idea generation, collaboration, and deep analysis in a mobile-first, low-friction environment – remains only partially addressed.

The feature set targeted in this capstone is not inherently complex and could be replicated easily.

The differentiation lies less in technical novelty and more in thoughtful integration, workflow design, and customisation to a specific niche industry user context.

In that sense, the project is less about building a defensible product at this stage, and more about assembling a coherent system that actually feels useful in practice.

As a beginner, my primary objective is to establish a functional foundation: understanding how to structure data, connect APIs, and present information in a way that supports investment decisionmaking. This creates a platform for future iteration and ideation.

Over time, the value can compound through refinement, user-specific tailoring, and potential network effects, rather than relying on any single feature to stand out.

Industry/ domain

Industry Backdrop

CONTEXT

Traditional Equity Investors	Competition	The Core Problem
3–5 year investment horizon	**Pods:** short-term momentum + tight risk control	**Macro** (geopolitics) + **factors** (industry, financial investment categories) **drive most returns, firm-specific a small driver**
Edge: patient capital + variable sizing	**Quants:** algorithm-driven factor trading	Traditional investors need patient capital to add investment value, but this requires **providing investors with valuable insights**
Outperformance comes in spurts — hard to consistently outperform	Both squeeze the traditional investor's edge	Yet **traditional investors waste too much time on commoditized firm-specific research that should be semi-automated**

- **What is the industry/ domain?**

Investment management listed equity research.

- **What is the current state of this industry? (e.g. challenges from startups)**

The equity research software landscape is crowded, with a mix of established platforms and a growing number of startups competing to deliver differentiated insights and workflows.

Costs can range from retail focused platforms $200/ yr to as high as $70/ yr for institutional grade systems and research.

Incumbent, seat-based business models are coming under increasing pressure from AI. The traditional logic of purchasing large numbers of licenses is being challenged. Firms can now rely on a smaller number of core subscriptions, with AI tools distributing, synthesising and repurposing that data across the organisation.

Our Stock Gossip Monitor capstone project takes this competitive ecosystem into account, with the long-term goal of developing an agent-driven system that can pull in data from multiple subscriptions, organise it coherently, and present it in a way that supports better investment decisions while reducing human cognitive load.

The anticipated end-state is closer to an agency-style “AI harness” model: highly customised to the user, and enhanced by network effects, where human industry specialists can contribute to research collaboratively, potentially in a pseudonymous format.

- **Is the project relevant to other industries?**

The idea of transforming large volumes of data and content into insights which experts find useful is broadly applicable across multiple industry verticals, not just investment research.

My advantage lies in a deep understanding of what truly matters within listed equity investing and the investment research process.

Stakeholders

Who are the stakeholders?

This is a single user app.

The stakeholders are investment research professionals such as myself who need to monitor stocks and their key fundamentals quickly.

Longer term targeted individuals are those in 1–5 person boutique teams, family offices and self-directed High Net Worth investors

What are the stakeholder's expectations?

1 **Simplicity** – alpha product must be easy to use. No onboarding required.

2 **Accurate & customizable** – underlying data must be accurate and if not accurate, easily overridden by the user.

3 **Well-documented** – product features must be fastidiously documented for future AI coder use, human developers as well as, in the long-term, execution by AI agents. All major AI prompts must be saved.

Product requirements: high level

DESIGN GOALS

01

Ease of Use

→ No onboarding required

→ Minimalist user interface

02

Accurate Relevant Data

→ **Relevant data** for each stock **must be accurate - irrelevant data** for each stock **will not be displayed**

→ **User overrides for most values, calculations for key values**

03

Heavy Documentation

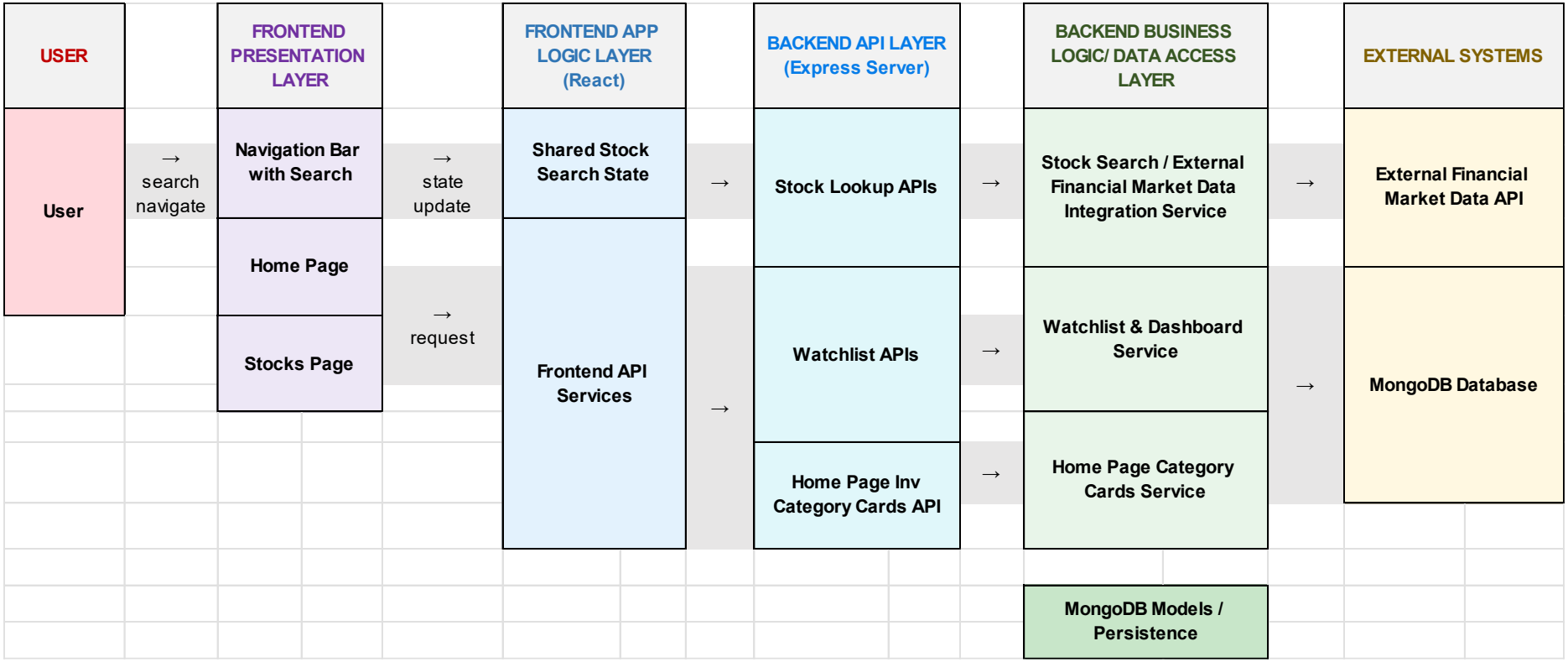
→ Prepare for future human developer/ AI coder input

→ DB schema, architecture and test/ script summary **markdowns should auto-update on code change**

Simple logic + accurate, customizable data + documentation = ready for future AI coder input and AI agent accessibility

Product Description

Architecture Diagram



Frontend Presentation Layer

The presentation layer is simply the part of the app the user can directly see and interact with.

It's the visible UI. Its job is to display information on the screen and capture user actions such as clicks, typing, and navigation. It does not own the main business rules, and it does not store permanent data in the database.

1. Navigation Bar with Search Box

This is the navbar that stays permanently at the top of the app. It gives the user a simple way to move between pages using the menu.

The navbar also contains the search box. When the user types a stock ticker such as AAPL, this part of the UI captures the typing and passes that input into the app's search flow.

2. Home page

This page displays the investment category cards. Each card shows the share price performance of stocks grouped into one investment category.

This helps the user see a high-level overview of how different groups of stocks have performed overall, rather than looking at one stock at a time.

So in simple terms, the Home page:

- displays category performance cards
- gives an overview of grouped stock performance
- focuses on summary information

Its main job is to present this information clearly to the user.

3. Stocks Page

This page displays the individual stock cards in the watchlist.

Each stock card shows details for one tracked stock, including:

- share price performance
- main table metrics
- more detailed metrics (if you click SHOW METRICS button)

This page is where the user moves from the high-level overview into the more detailed stock-by-stock view.

So in simple terms, the Stocks Page:

- displays each saved watchlist stock
- shows detailed stock performance and metrics
- helps the user inspect one stock at a time more closely

Frontend Logic Layer (React)

This is the React app running in the user's browser. It has two big functions, to manage frontend state (Shared Stock Search State), and to communicate with the backend ('frontend API services').

React sits between:

- the Presentation layer, which shows the visible user interface; &
- the Backend layer, which runs on the server and handles data, rules and storage

Another way to think of React is the frontend logic behind the visible UI that is the Presentation layer.

It is important to note the React app does not talk directly to MongoDB or the external financial market data API. Instead, it sends HTTP requests to the Express backend, usually through routes under /api/*. The backend then decides what to do next, such as reading from MongoDB or calling the external financial market data API.

1. Shared Stock Search State

State means information the app is remembering right now while the user is using it.

For example, if the user types "Apple" in the search bar, the app may need to remember:

- what the user typed
- what search results came back
- whether the search is still loading
- whether an error happened
- which stock the user selected

This remembered information is called state.

Frontend state is usually temporary. If the page is refreshed, that state is often reset unless it has been saved somewhere else.

Shared state means that more than one React component can use the same information.

For example:

- the navigation bar can update the search text
- the shared search results component can display the matching stocks
- the Stocks page can react to the selected stock

Because they are all using the same shared state, they stay in sync automatically.

So in simple terms:

- state = information the app is currently remembering
- shared state = information that multiple components can read and update together

Shared Stock Search State is the React app's shared memory for stock search and watchlist-related actions.

This app really needs shared stock search state for 3 key reasons:

a. The search bar is in the navbar, but the search result is shown in both pages

The user types into the search box at the top of the app in the navbar. But the search results are not only shown in the search box. Rather, the Home page and the Stocks page can both show the shared search results panel.

So one part of the app captures the search input. Another part of the app displays the results.

Those two parts need shared access to the same search information.

b. The app needs one consistent search flow

For example:

- user types AAPL in the navbar
- search runs
- results appear
- user clicks ADD STOCK or SEE STOCK
- Stocks page reacts correctly

That search flow crosses multiple components and even multiple pages.

Shared state makes that possible without passing data manually through lots of unrelated components.

c. The app also shares watchlist-related search behaviour

This app does not just search stocks. It also needs to know things like:

- is this stock already in the watchlist?
- should the button say ADD STOCK or SEE STOCK?
- after clicking a result, should the app open an existing stock or import a new one?

That logic needs to be shared too.

2. Frontend API services

This is the part of React that talks to the backend API using HTTP requests and receives data back as JSON. You can think of it as the frontend's messenger to the backend.

It sends HTTP requests to do things like:

- load homepage category cards
- load watchlist dashboards
- update dashboard data
- refresh stock data

React components should not all contain raw API request code. If every component made its own HTTP calls directly, the same request logic would be repeated in many places. That makes the code repetitive, harder to maintain, and more difficult to update when an API route or JSON response format changes. It also mixes UI code with data-fetching code, which makes components harder to read and understand.

This problem is solved by using frontend API service files. These files act like helpers that know:

- which backend URL to call
- which HTTP method to use, such as GET, POST, or PATCH
- what data to send in the request
- how to handle and shape the JSON response

This keeps React components more focused on displaying the UI, while the API service files handle communication with the backend.

Backend API (Express) Layer

The backend in this app runs on a Node.js / Express server. You can think of it as the server-side part of the system that handles requests, applies backend logic, and returns data to the frontend.

In this app, the Backend API is mainly about routes because routes are the backend's entry points. They are the specific URLs the React frontend can call when it needs something done. For example, the frontend might need to:

- search for a stock
- load watchlist data
- refresh a stock
- load Home page category cards

To do all these things the frontend sends an HTTP request to a backend route. The backend route receives that request and returns a JSON response.

This is why routes are so important in the Backend API layer:

- they define what jobs the backend offers
- they define which URL the frontend calls
- they define which HTTP method is used, such as GET, POST, PATCH, or DELETE
- they define where the request enters the backend system

The user does not interact with these routes directly in normal use. Instead, the user clicks buttons and types into the browser UI, and then the React frontend sends HTTP requests to the backend on the user's behalf.

Definitions: an API route is a backend URL that performs one specific job when the frontend sends it an HTTP request. A path *part like :ticker* is a variable part of the route. For example, in */api/stock-prices/:ticker*, the *:ticker* part could become AAPL which denotes the stock Apple.

An important beginner idea is this:

- the frontend does not talk directly to MongoDB
- the frontend does not talk directly to external financial market data API
- all of that goes through the Express backend, usually through routes under /api/*

There are 3 main categories of API routes in this app: Stock Lookup API routes, Watchlist API routes and a single Home page Category API route.

Finally, one important software lesson is: API routes are just the entry points. Services do most of the business work behind those routes.

1. Stock Lookup API routes

These are the backend routes used to find stock information. They are read-only, which means they look up data but do not create saved watchlist records.

This route group is the one most closely connected to external market data lookup.

GET /api/stocks/search
Searches for a stock by company name or ticker and returns JSON search results.

GET /api/stock-prices/:ticker
Returns JSON price data for one stock ticker.

2. Watchlist API routes

These routes handle the stocks that this app saves, tracks, updates, customizes, and refreshes in the watchlist.

This is the backend route group for the main watchlist stock-monitoring part of the app.

Import / create routes
POST /api/watchlist/import
Imports a stock into the watchlist by fetching stock data and creating a saved watchlist record.

POST /api/watchlist/
Creates a new watchlist stock record manually.

Read / load routes

GET /api/watchlist/summary
Returns a lightweight summary list of watchlist stocks for shared search and navigation state.

GET /api/watchlist/dashboards
Returns batched dashboard data so the Stocks page can load many stock cards more efficiently.

GET /api/watchlist/
Returns all saved watchlist stock records.

GET /api/watchlist/:ticker
Returns one saved watchlist stock record for the given ticker.

GET /api/watchlist/:ticker/metrics-view
Returns the detailed metrics-view data for one stock card.

Update / customize routes

PATCH /api/watchlist/:ticker
Updates saved watchlist stock details, such as investment category or company name.

PATCH /api/watchlist/:ticker/annual/:fiscalYear/overrides
Updates saved user override values for one annual fiscal-year field on a stock.

PATCH /api/watchlist/:ticker/forecast/:bucket/overrides
Updates saved user override values for one forecast bucket, such as fy1, fy2, or fy3.

PATCH /api/watchlist/:ticker/metrics/overrides
Updates saved user override values for top-level stock metrics.

PATCH /api/watchlist/:ticker/metrics-row-preferences
Saves display preferences for stock-card rows, such as whether a row is shown or bold.

Delete route

DELETE /api/watchlist/:ticker
Removes a stock from the saved watchlist.

Refresh route

POST /api/watchlist/:ticker/refresh
Refreshes a saved stock by pulling in newer backend-supported stock data while keeping user changes where possible.

3. Home page Category API route

This route powers the Home page investment category cards.

Its job is not to directly act like an external API. Rather, it acts as an internal backend endpoint layer for the Home page.

The Home page sends an HTTP request. This API route receives the request. It passes the work to the Homepage Category Cards Service. That service then loads data, mainly from MongoDB, and sometimes from the external financial market data API if needed. The backend returns JSON back to the frontend.

POST /api/homepage/investment-category-cards/query
Returns JSON category-card data for the Home page.

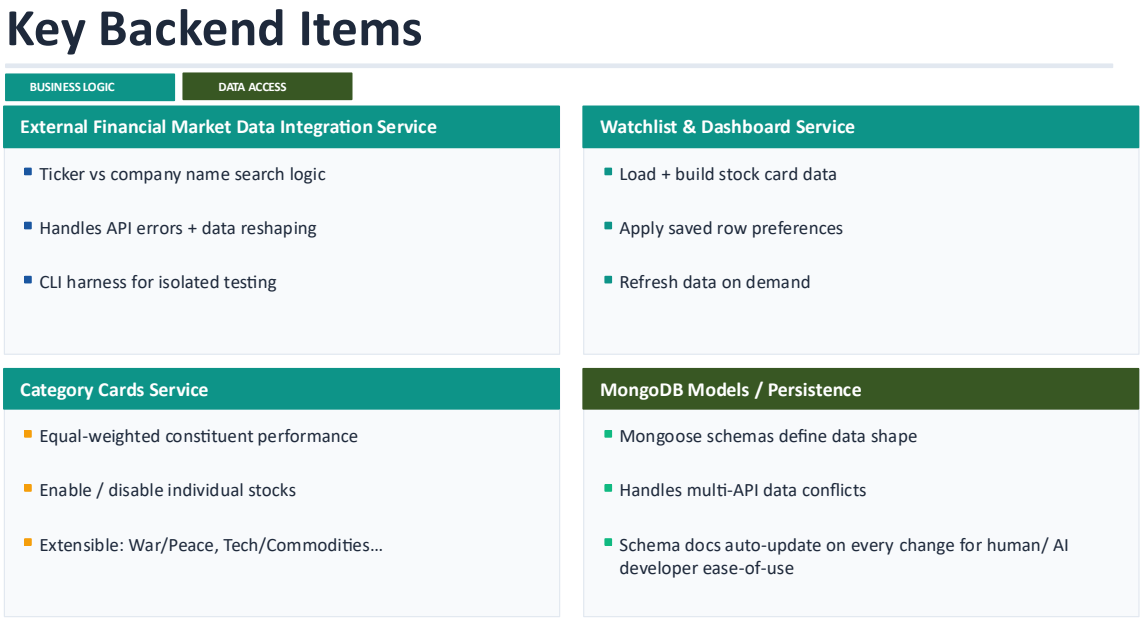
Backend Business Logic/ Data Access Layer

This is the part of the backend where most of the application's real work happens.

The backend API layer receives HTTP requests from the frontend, but it does not do most of the real processing itself. Instead, it passes the work into this backend business logic/ data access layer. It's the layer which thinks, calculates, fetches, saves, and prepares the data the app needs.

In this particular app, the work can be split into four main parts:

- **Stock Search / external financial market data API Integration Service** = handles stock lookup and talks to the external financial market data API.
- **Watchlist & Dashboard Service** = handles tracked stocks and stock dashboard data
- **Homepage Category Cards Service** = builds the grouped card data for the Home page
- **MongoDB Models / Persistence** = defines and stores the app's permanent saved data



1. Stock Search / external financial market data API Integration Service

This part handles stock searching and market-data lookup.

For example, if the user searches for Tesla or TSLA, this service helps turn that search into real stock results. It knows how to talk to the external financial market data API, including:

- which financial market data API endpoint to call
- what request format to use
- how to handle API errors
- how to reshape the response into something the rest of this app can use

So in this app, this service is the backend part that connects stock searching to real outside market data.

2. Watchlist & Dashboard Service

This part handles the saved stocks in the watchlist and the stock dashboard data shown on the Stocks page.

In this app, it helps with things like:

- loading saved watchlist stocks
- building dashboard data for stock cards
- loading summary data for shared search/navigation
- using saved row preferences
- refreshing dashboard-related stock data when needed

This service mainly works with the app's own saved data in MongoDB, and in some cases it also uses external financial market API backed price data to help build stock dashboard information.

3. Homepage Category Cards Service

This part handles the data needed for the Home page investment category cards.

In this app, the Home page does not just show one stock. It shows grouped category cards, such as investment categories that summarize how groups of stocks have performed.

This service helps by:

- loading the relevant saved stock/category data
- applying constituent preferences
- building the final card data the Home page needs
- sometimes using external financial market data API price data if needed for card performance calculations

4. MongoDB Models and Persistence

This part of the app is about permanent saved data, which is usually called persistence. Before that, it helps to know what Mongoose is.

Mongoose is a tool the backend uses to work with MongoDB more easily. MongoDB is the database where this app stores permanent data, and Mongoose lets the developer define clear data models for that database.

You can think of it like this:

- MongoDB = the place where the saved data lives ('persists')
- Mongoose models = the blueprints that describe how that data should look

Persistence can be contrasted with state, which was handled earlier in the React layer. Here is the difference between state and persistence:

- state = temporary information the app remembers right now while it is running. It is like your RAM.
- persistence = information saved permanently so it is still there later. It is like your SSD or hard drive.

For example:

- if you type in a search box, that is usually state
- if you save a stock into the watchlist, that becomes persistent data

In this app, the MongoDB Models / Persistence layer is the backend code that defines:

- what data gets saved
- what shape that data must have
- how the backend reads and writes it

Specifically, the MongoDB models define things like:

- saved watchlist stocks
- saved row display preferences
- saved homepage category constituent preferences
- cached price-history data
- saved lens or category display rules

External Systems

These are systems that exist outside the main app code, but the app still depends on them to work properly.

They are called external systems because they are not part of the React frontend logic or the Express backend logic itself. Instead, the app connects to them when it needs data or storage.

1. External Financial Market Data API

This app does not keep its own giant global database of live stock market information such as search results, company lookup data, or market price history.

Getting access to that kind of financial dataset would be extremely difficult and expensive. Instead, this app calls the External Financial Market Data API when it needs outside market data.

So in simple terms:

- the app asks the external financial market data provider for real financial market data
- external financial market data provider sends that data back
- the app uses it to support stock search, stock lookup, and some price-related features

You can think of it as our app does not own this market data. Rather, it is “borrowing access” to the data through this external financial market data API.

2. MongoDB Database

MongoDB is the database server where this app stores its own permanent data.

Unlike the external financial market data API, which provides outside market data, our app’s MongoDB stores data that belongs to this app itself.

For example, MongoDB stores things like:

- saved watchlist stocks
- saved row display preferences
- saved homepage category constituent preferences
- cached price-history data
- saved lens or category display rules

User Stories

1. Search and watchlist navigation

These user storis are mainly about:

- finding stocks
- moving between search and watchlist flows
- managing which stocks are in the watchlist
- keeping the watchlist usable and responsive

#	User Story Title	User Story Description	Priority	Additional Notes
1	Shared stock search	I want to search for a stock by ticker or company name from the navbar so I can quickly find stocks from anywhere in the app.	1	Search is shared across Home and Stocks.
2	Search results – add a new stock	I want search results to show ADD STOCK for stocks not already in my watchlist so I can import them directly into the app.	1	The app distinguishes new stocks from existing ones.
3	Search results – open an existing stock	I want search results to show SEE STOCK for stocks already in my watchlist so I can jump straight to the existing stock card instead of adding a duplicate.	1	This is one of the clearest app-specific flows.
4	Stocks page – view watchlist stock cards	I want a Stocks page that shows one card per saved stock so I can monitor each stock's share-price performance and summary metrics.	1	This is the main watchlist page.
5	Stocks page – load large watchlists progressively	I want the Stocks page to load quickly even when many stocks are saved so I can start using the page without waiting for every stock card to fully load.	2	This is a strong app requirement, even if it is more technical than visible.
6	Stock card – remove a stock from the watchlist	I want to remove a stock from my watchlist with a confirmation step so I can clean up my list without accidentally deleting a stock.	1	Confirmation dialog is part of the current flow.
7	Stock card – preserve user data during refresh	I want stock data to refresh in the background without wiping my manual changes so I can benefit from newer backend data while keeping my own edits.	1	This is a real design goal of the refresh system.

2. Home page category cards

These are grouped here because they are mainly about:

- analysing category-level performance
- understanding which stocks belong to each category
- controlling category composition for aggregate analysis

#	User Story Title	User Story Description	Priority	Additional Notes
8	Home page – view investment category cards	I want a Home page that shows investment category cards so I can quickly see how each category has performed over time on an indexed chart.	1	Category cards are a core landing-page feature.
9	Home page – view category constituents	I want to open a category card and view its constituent stocks so I can understand which companies are contributing to that category.	1	Constituents appear inside the category-card experience rather than on a separate page.
10	Home page – enable or disable a constituent	I want to disable or re-enable a constituent within a category so I can control which stocks are included in the category's aggregate chart.	2	This is a real persisted user-preference feature.

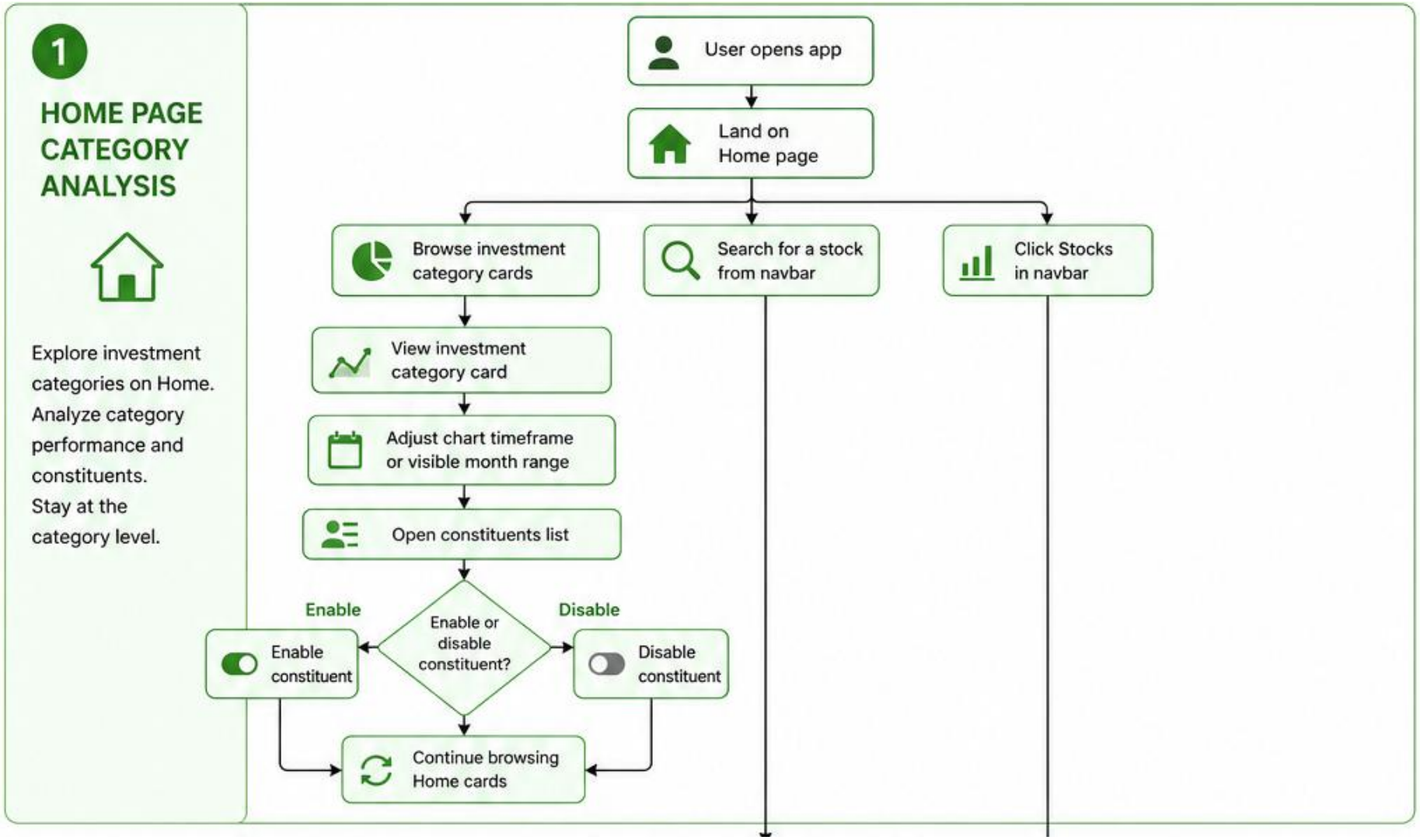
3. Individual stock card customisation

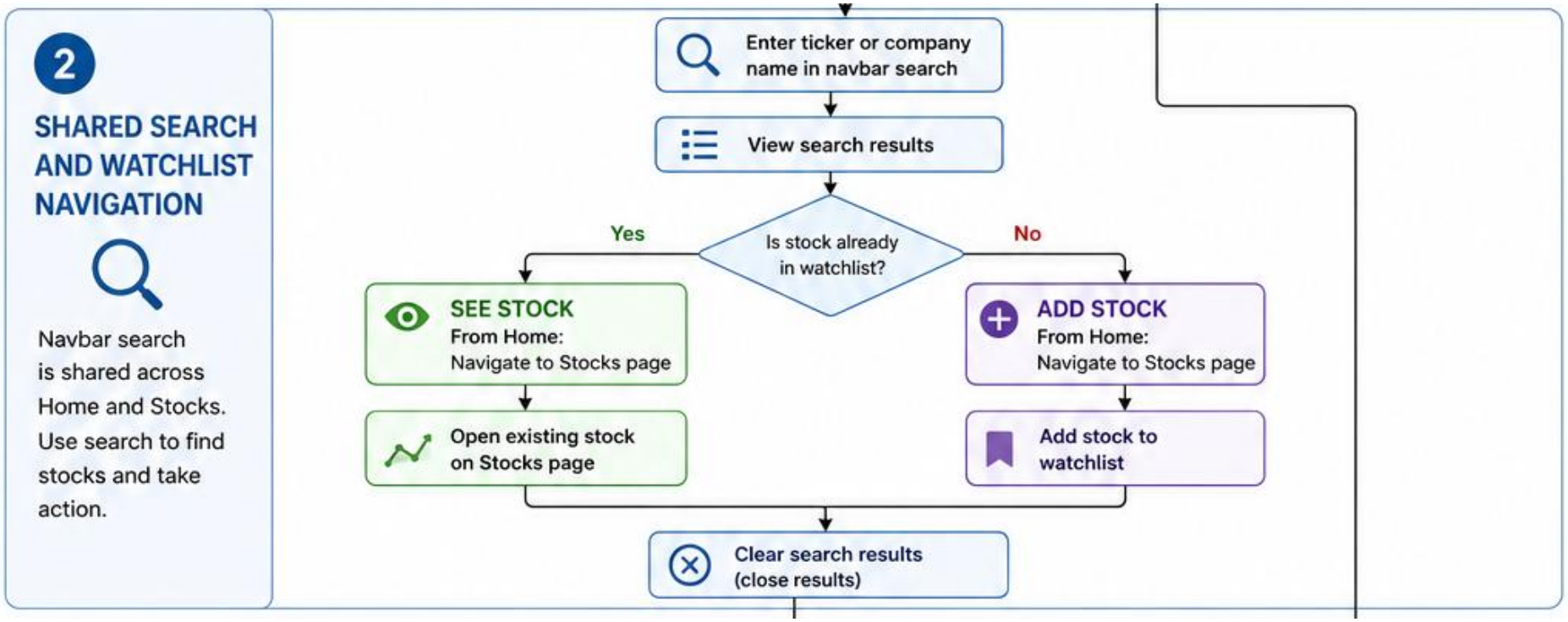
These stories are grouped together because they are mainly about:

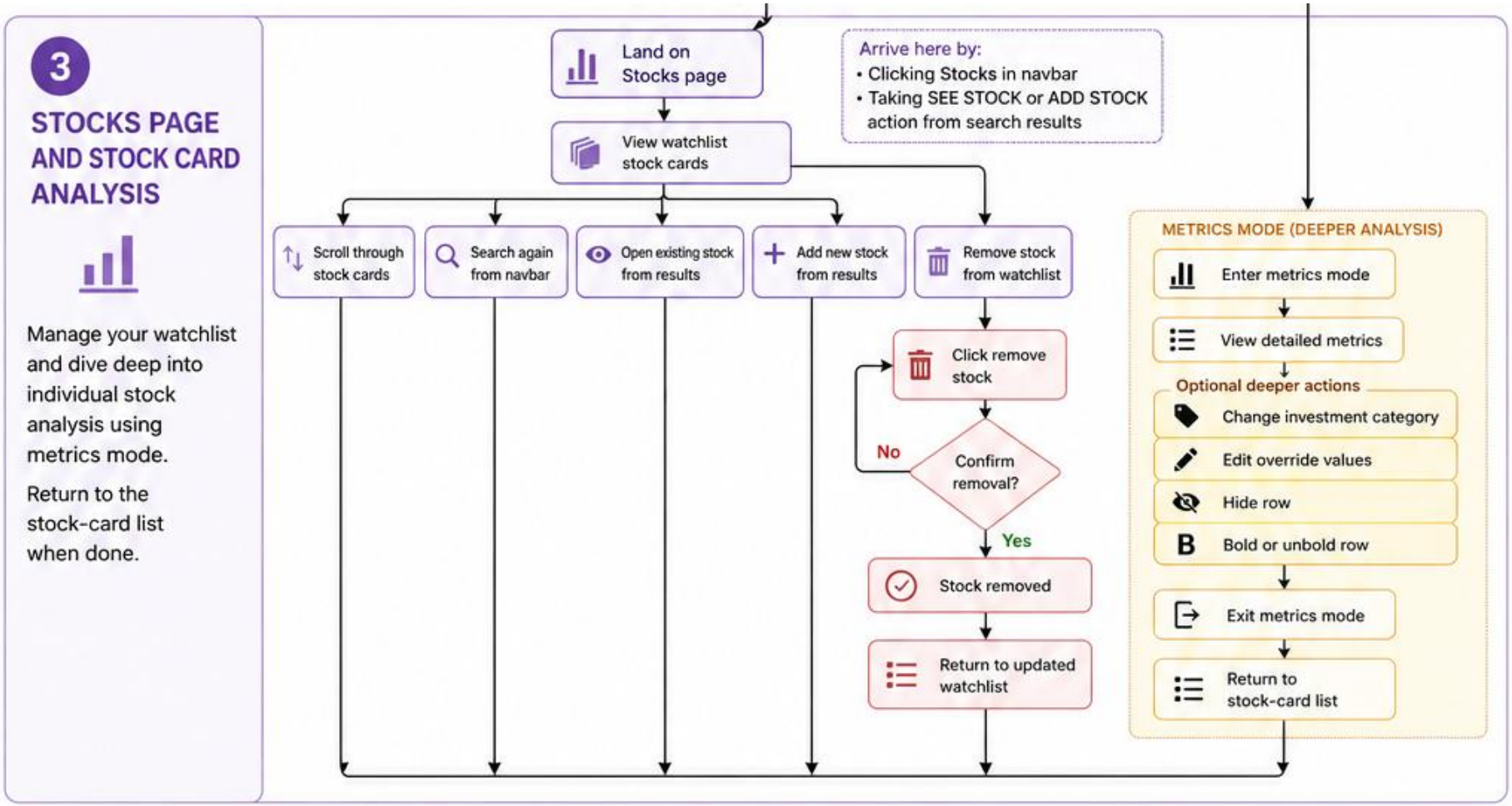
- drilling deeper into one stock
- customising how stock information is shown
- tailoring stock cards to the user's own analysis style

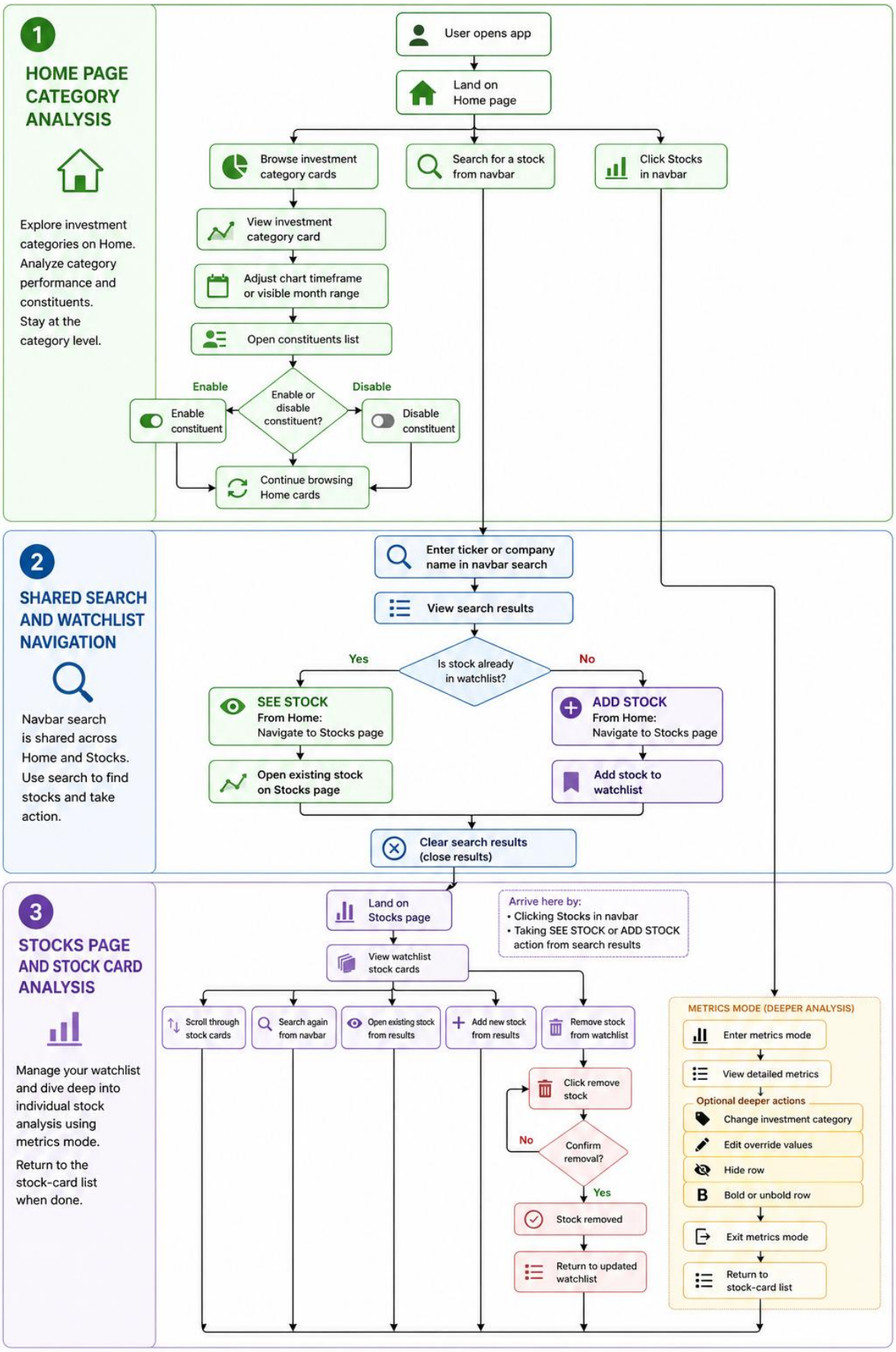
#	User Story Title	User Story Description	Priority	Additional Notes
11	Stock card – open focused metrics mode	I want to open a focused metrics view for one stock so I can study its detailed financial metrics without distraction from other cards.	1	The page hides sibling cards while one stock is in focused metrics mode.
12	Stock card – view detailed metrics lazily	I want detailed metrics to load only when I ask for them so the page stays faster while still letting me inspect deeper information when needed.	2	This is part of the actual UX and performance design.
13	Stock card – change investment category	I want to assign or change a stock's investment category so the stock is grouped correctly within the app's category logic.	2	There is an investment-category UI/control on stock cards.
14	Stock card – edit override values	I want to override certain stock values manually so I can adjust annual, forecast, or top-level metrics when I want my own assumptions reflected.	1	Overrides are a major feature in this app.
15	Stock card – hide a detailed metrics row	I want to hide a detailed metrics row I do not care about so I can simplify the stock card and focus on the rows that matter to me.	1	Available through the row action menu.
16	Stock card – bold important rows	I want to bold important rows in the main table or detailed metrics so I can visually highlight the numbers I care about most.	1	The app supports both BOLD and UNBOLD.
17	Stock card – keep row preferences saved	I want my hide/show and bold/unbold row preferences to persist so I do not have to re-customise the same stock card every time.	1	This is backed by saved row-preference data.
18	Charts – keep hover information readable	I want chart hover tooltips to stay readable near the chart edges so I can inspect values without the tooltip getting clipped off-screen.	2	Present on both category and stock charts.

User Flow


1
HOME PAGE CATEGORY ANALYSIS
Explore investment categories on Home. Analyze category performance and constituents. Stay at the category level.
User opens app
Land on Home page
Browse investment category cards
Search for a stock from navbar
Click Stocks in navbar
View investment category card
Adjust chart timeframe or visible month range
Open constituents list
Enable or disable constituent?
Enable
Disable
Enable constituent
Disable constituent
Continue browsing Home cards


2
SHARED SEARCH AND WATCHLIST NAVIGATION
Navbar search is shared across Home and Stocks. Use search to find stocks and take action.
Enter ticker or company name in navbar search
View search results
Is stock already in watchlist?
Yes
No
SEE STOCK
From Home:
Navigate to Stocks page
Open existing stock on Stocks page
ADD STOCK
From Home:
Navigate to Stocks page
Add stock to watchlist
Clear search results (close results)


3
STOCKS PAGE AND STOCK CARD ANALYSIS
Manage your watchlist and dive deep into individual stock analysis using metrics mode.
Return to the stock-card list when done.
Land on Stocks page
Arrive here by:
• Clicking Stocks in navbar
• Taking SEE STOCK or ADD STOCK action from search results
View watchlist stock cards
Scroll through stock cards
Search again from navbar
Open existing stock from results
Add new stock from results
Remove stock from watchlist
Click remove stock
Confirm removal?
No
Yes
Stock removed
Return to updated watchlist
METRICS MODE (DEEPER ANALYSIS)
Enter metrics mode
View detailed metrics
Optional deeper actions
Change investment category
Edit override values
Hide row
Bold or unbold row
Exit metrics mode
Return to stock-card list


1
HOME PAGE CATEGORY ANALYSIS
Explore investment categories on Home. Analyze category performance and constituents. Stay at the category level.
User opens app
Land on Home page
Browse investment category cards
Search for a stock from navbar
Click Stocks in navbar
View investment category card
Adjust chart timeframe or visible month range
Open constituents list
Enable
Disable
Enable or disable constituent?
Enable constituent
Disable constituent
Continue browsing Home cards
2
SHARED SEARCH AND WATCHLIST NAVIGATION
Navbar search is shared across Home and Stocks. Use search to find stocks and take action.
Enter ticker or company name in navbar search
View search results
Yes
No
Is stock already in watchlist?
SEE STOCK
From Home: Navigate to Stocks page
ADD STOCK
From Home: Navigate to Stocks page
Open existing stock on Stocks page
Add stock to watchlist
Clear search results (close results)
3
STOCKS PAGE AND STOCK CARD ANALYSIS
Manage your watchlist and dive deep into individual stock analysis using metrics mode.
Return to the stock-card list when done.
Land on Stocks page
Arrive here by:
• Clicking Stocks in navbar
• Taking SEE STOCK or ADD STOCK action from search results
View watchlist stock cards
Scroll through stock cards
Search again from navbar
Open existing stock from results
Add new stock from results
Remove stock from watchlist
Click remove stock
No
Confirm removal?
Yes
Stock removed
Return to updated watchlist
METRICS MODE (DEEPER ANALYSIS)
Enter metrics mode
View detailed metrics
Optional deeper actions
Change investment category
Edit override values
Hide row
Bold or unbold row
Exit metrics mode
Return to stock-card list

Wireframe Design

Initial low-fidelity “design”

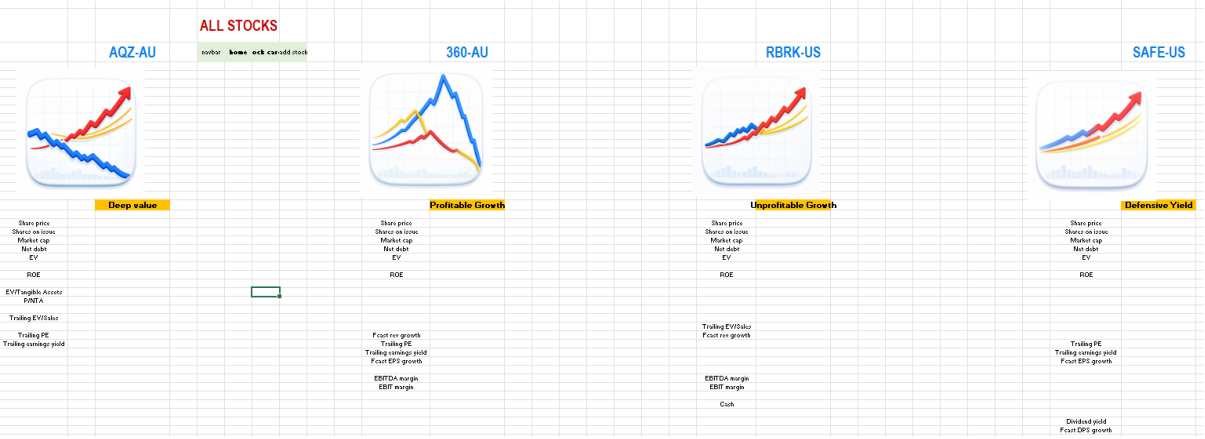


Table mid-fidelity wireframe (equivalent to Mini Project 1)

With AI now making frontend development much easier, it’s often better to ship a rough, semi-functional frontend logic for yourself and iterate from there, rather than over-optimising static wireframes upfront.

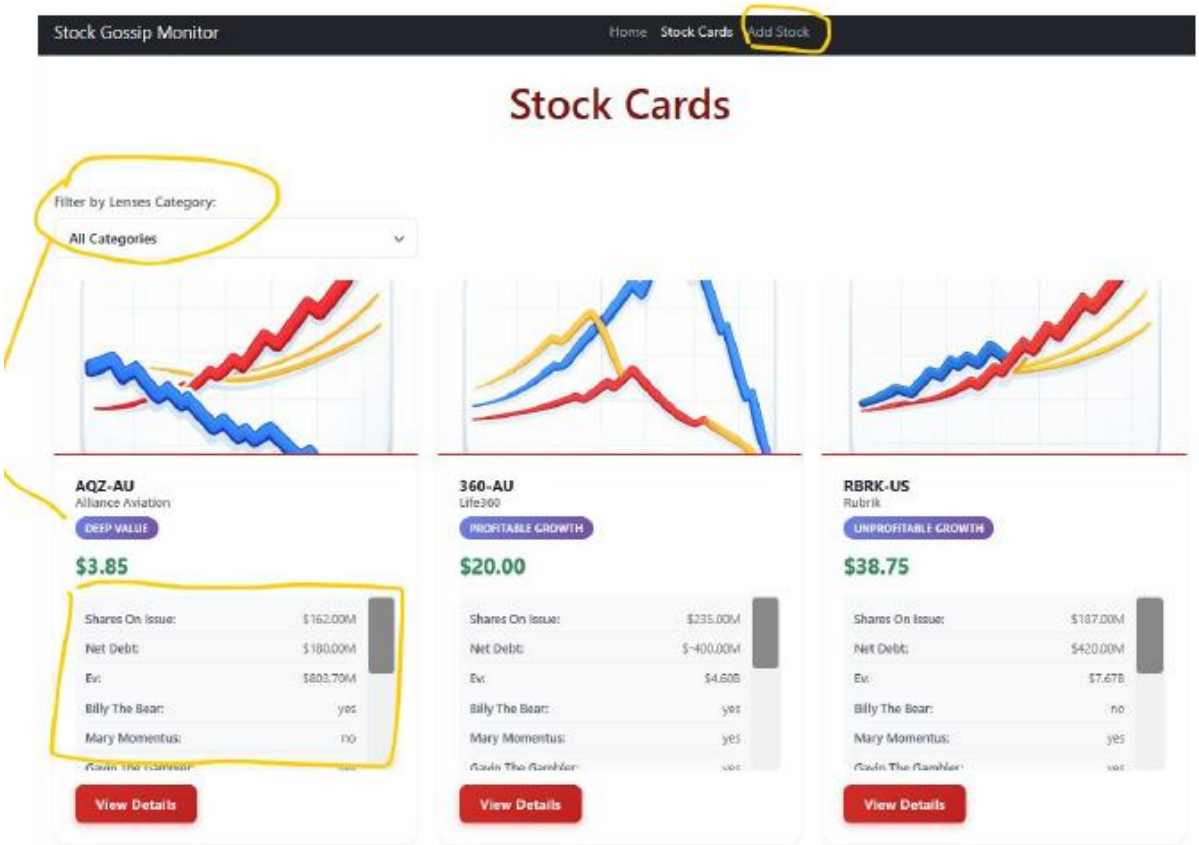
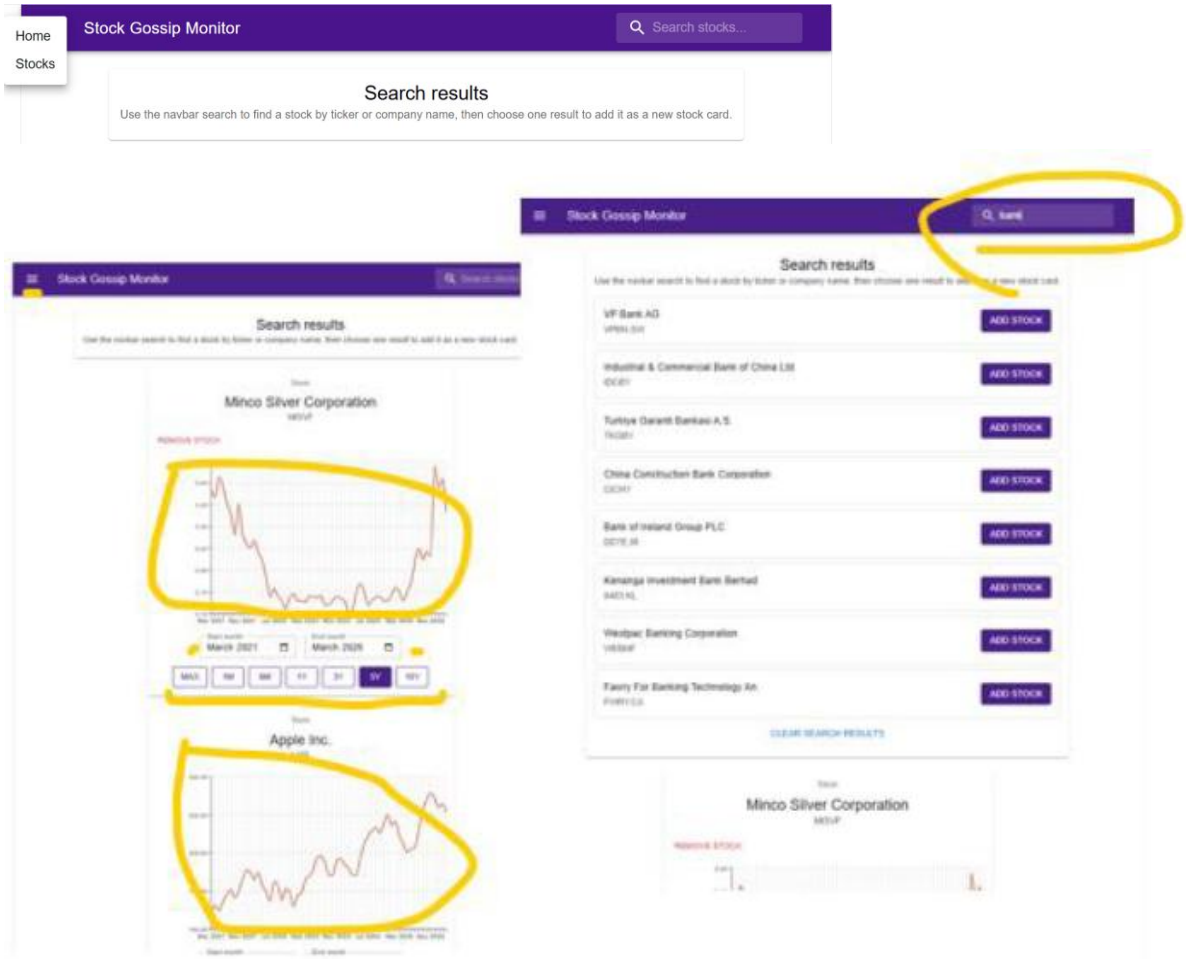


Chart hi-fidelity wireframe (equivalent to Mini Project 2)

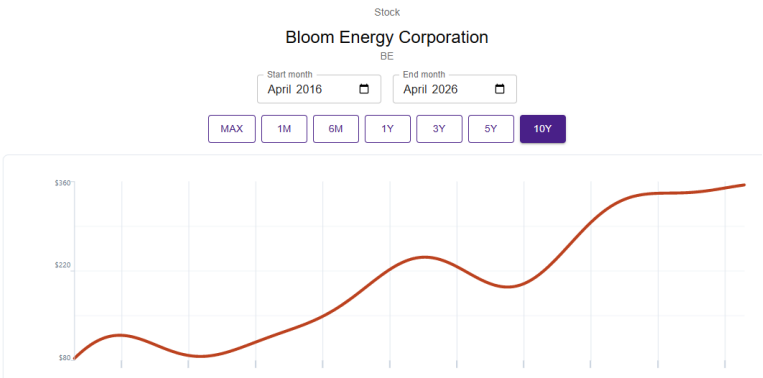


Consolidated wireframe

I took my simple design from Mini Project 2 and added a table at the bottom. At this stage, clean UI is more important to me than aesthetics.

	Dec 2021 Y	Dec 2022 Y	Dec 2023 Y	Dec 2024 Y	Dec 2025 Y
Average Price	25.75	20.02	16.54	13.31	49.32
Basic Weighted Avg Shares	173	186	213	227	240
Market Capitalization	3,803	3,555	3,148	5,050	20,889
Net Debt	131	63	182	326	164
Enterprise Value Average	5,237	4,433	4,327	3,776	12,418
Cash & Equivalents per Sha...	2.28	1.87	3.12	3.53	10.21
Tangible Book Value per Sh...	-0.02	2.04	2.45	2.57	3.3
Book Value per Share	-18.81	-19.17	-18.18	-17.14	-16.58
Total Debt/Total Assets (%)	65.11	52.43	60.27	57.57	68.05
Revenue per Share	5.61	6.45	6.27	6.48	8.42
Diluted EPS from Cont Ops	-1.11	-1.69	-1.45	-0.12	-0.36
Free Cash Flow per Basic S...	-0.64	-1.66	-2.15	0.15	0.24
Dividend per Share	- -	- -	- -	0.01	- -
Operating Income per Share	-0.66	-1.4	-0.98	0.1	0.3

Below, I implemented a custom React component using Figma Make to replace the MUI X chart from Mini Project 2. This was a last-resort solution to resolve a data column versus timeline alignment issue driven by a specific product requirement.



FY end	Dec 2016	Dec 2017	Dec 2018	Dec 2019	Dec 2020	Dec 2021	Dec 2022	Dec 2023	Dec 2024	Dec 2025
Average Price	$119.33	$87.85	$108.68	$149.04	$222.45	$226.89	$200.03	$295.44	$341.68	$349.98
Basic Weighted Avg Shares	173	173	173	173	173	173	173	173	173	173
Market Capitalization	20.6K	15.2K	18.8K	25.8K	38.5K	39.3K	34.6K	51.1K	59.1K	60.5K
Net Debt	797.9	873.9	957.3	1K	1.1K	1.3K	1.4K	1.5K	1.6K	1.8K
Enterprise Value Average	21.4K	16.1K	19.8K	26.8K	39.6K	40.5K	36K	52.6K	60.7K	62.3K

Out of Scope

Features next in line for development

The below items are immediate things to work on post Capstone completion:

- Add horizontal scroll for Home Page Inv Category Cards
- Stocks page filter by investment category
- User re-order rows (drag & drop)
- User re-order stock cards (drag & drop)
- Add appropriate currency symbol to Y axis depending on currency of shares

Medium term features

- Ability to select and copy scrollable table as text
- User Login
- Separate earnings and revenue revisions table
- Get Total Shareholder Return metric
- Show TSR CAGR in preset data range
- EPS historical CAGR field value, DPS historical CAGR field value
- ROE/ ROIC metric field value
- User-generated investment categories and new user-generated categories (e.g. industry, country)
- User can customise lenses for each investment category
- Multiple user accounts each with their own row view/ bold-unbold preferences
- Multiple financial market data APIs and the their impact on user overrides
- Inclusion of exchange announcements, large broker reports to explain firm-specific price moves
- Industry newsflow to explain sector price moves

- Hook up to AI agent so they can fill in user overrides for me based on a variety of data sources

- Introduction of portfolio weights and portfolio tracking so you can see the raw fundamentals of your portfolio of stocks, in aggregate.

Non-functional Requirements

Ease of use

Ease of use, for the benefit of both human users as well as a future AI agent, is a top priority.

This is not an app that should require an onboarding session for human users.

In practice this means a minimalist feature set, clear visual hierarchy, predictable page structure, explicit labels, and reusable interaction patterns across Home, Stocks, and the shared search flow.

It also means reducing unnecessary decisions for the user, keeping the navigation simple, using direct action wording such as ADD STOCK, SEE STOCK, and Remove stock, and making important flows easy to understand at a glance.

The same principle has also shaped the technical design: lightweight shared search state, dedicated frontend API helpers and clearly separated backend route groups make the system easier for a future AI agent to interpret, navigate, and act on safely without needing hidden assumptions or project-specific tribal knowledge.

Speed

Speed was not considered during the planning stage. However speed of loading issues were quickly uncovered once the core feature set was built and tested. This was solved by loading less data up front, batching what must be loaded, delaying what is not immediately needed, refreshing in the background and caching repeated expensive work

Reducing the amount of data loaded on first render

Instead of loading full stock-card data everywhere, the app was split into lightweight shared state and page-heavy data. A new GET /api/watchlist/summary route now returns only the small watchlist payload needed for shared search state, SEE STOCK detection, and add/open/remove flows. Then, only when the user visits the Stocks page, the app requests the heavier stock-card data through a separate batched bootstrap endpoint, GET /api/watchlist/dashboards. This avoids unnecessary over-fetching and reduces the amount of JSON transferred during initial page load.

Batching requests

As originally designed, the browser risked making one dashboard request per stock card, which did not scale well as the watchlist grew.

This was replaced with a single batched request that loads the first-paint dashboard payload for many stocks at once. In beginner terms, this means the browser now asks for many cards in one trip instead of making many small trips back and forth to the server.

Lazy loading

As originally designed, all metrics were loaded immediately for a stock card, yet only the main table was shown. This was not a sustainable arrangement.

Now detailed metrics data is no longer loaded immediately for every stock. Instead, GET /api/watchlist/:ticker/metrics-view is only called when the user actively opens the metrics section for a stock. This is a classic lazy-loading optimisation: expensive data is deferred until it is actually needed.

The app also uses React route-level lazy loading for pages, so large page bundles such as Home and Stocks are only downloaded when the user navigates to them.

Non-blocking background refresh

Stocks that were added to the watchlist before newer fields were introduced may still require a refresh so their saved data matches the latest backend data structure. However, POST /api/watchlist/:ticker/refresh is now run in the background after the initial page render, rather than blocking the entire Stocks page before anything is shown.

In practical terms, this means the app now prioritises first paint and perceived performance: the user sees the page quickly, while older watchlist records are upgraded quietly in the background instead of delaying the whole screen.

Viewport-based progressive rendering

On the Stocks page, stock cards use IntersectionObserver so cards are only fully activated when they are near the visible part of the page. This reduces the amount of expensive chart and metrics work done immediately for off-screen cards and improves perceived performance when many stocks are in the watchlist.

Removing duplicate requests

On the Home page, the homepage category-card route POST /api/homepage/investment-category-cards/query now returns the canonical trailing 5-year payload on the first request.

That way, the Home page no longer mounts and then immediately re-queries the same data. This removes a wasted round-trip and speeds up first load.

Caching

Price history for investment category cards shown of the Home page is now cached. This means the backend stores compact monthly price history in StockPriceHistoryCache, which avoids repeatedly fetching and condensing the same historical market data from the external financial market data API for category card calculations.

Basically, the app remembers previously prepared price-history data so it does not have to rebuild it every time.

Reliability

The application needs to be highly reliable. However, because this is an early version built by a beginner learner using AI coding tools, it is realistic to expect that some bugs will only be discovered over time. Reliability will therefore be improved progressively through debugging, refinement and continued expansion of the automated test suite.

Security

Features such as user login, storage of personal details, inactivity timeout and data encryption have not been included in this project. This is because the app was built as a beginner learner project, with the main focus placed on core functionality, architecture and data flow rather than production-level security design.

However, one rudimentary security-related step has been taken: the .env file is included in .gitignore. This prevents sensitive values such as API keys from being committed to GitHub by mistake.

Performance & scalability

Given the developer is a beginner, detailed performance engineering topics such as precise CPU usage, RAM usage, request throughput, and formal server response-time benchmarking were not initially measured in a systematic way in this project.

Instead, my performance work in this application focused first on the most obvious bottlenecks that appeared during development, especially once the watchlist grew beyond a small set of stocks and page loading began to slow noticeably.

A final two-part large-watchlist performance test was later added to investigate whether the application could scale well beyond normal classroom-sized usage (i.e. a watchlist of around 30 stocks.)

This consisted of:

- a custom backend performance harness, which measures route latency, payload size, heap memory growth, RSS memory growth and repeated-run drift across large seeded watchlist datasets
- a real-browser Playwright benchmark, which measures user-facing browser behaviour such as first visible shell render, first visible real dashboard render, first usable interaction, progressive card activation during scrolling and browser memory growth

This performance test differs from the rest of the test suite because its purpose is not to verify functional correctness. Rather, it is designed to test scalability and resource behaviour under heavy load using deterministic seeded datasets such as 100, 500, 1000 and 2000 stocks.

It also sits outside the normal lightweight automated test flow because benchmarking thousands of stock cards is much more resource-intensive than ordinary unit, integration or CRUD tests.

One important finding was that the original large-watchlist seed process itself became a memory bottleneck. The backend initially failed at higher stock counts because the test seeder built the full synthetic dataset in memory before inserting it into MongoDB. This caused heap blow-ups before the real route benchmarking could even begin. That problem was resolved by refactoring the performance seeding system into a chunked / batched insertion process, so stocks, row preferences, constituent preferences, and cached price-history records are now generated and inserted in smaller batches rather than as one giant in-memory array.

The frontend also failed at scale, but for different reasons. The Stocks page originally waited too long before showing useful content, and the browser benchmark revealed several related problems:

- the page could wait on too much stock-card bootstrap work before first paint
- background refresh could begin too early and race ahead of the first real visible dashboard
- too many stock cards could activate too early on initial layout, leaving no meaningful progressive activation for scrolling

These frontend issues were painstakingly addressed through a series of performance-focused changes:

- rendering lightweight summary shells first instead of blocking on full stock-card dashboard payloads
- loading dashboard bootstrap data through a small queued chunking system rather than one giant whole-watchlist step
- delaying legacy background refresh until after the first real dashboard had visibly painted
- adding a render window so the page only mounts the part of the watchlist the user is near
- making shell cards reserve roughly the same vertical space as real dashboards, so first-paint layout more closely matches final scrolling layout
- tightening the viewport observer margin so only cards genuinely near the viewport activate on first paint

Based on the updated backend and frontend performance harness, I believe this app in its current form can reliably scale to at least **2,000 stocks** in the benchmark environment. The latest real-browser benchmark also completed successfully across the larger seeded scenarios used in the performance toolkit. **Offsetting this, however, were very minor speed degradations noticed by the user (even at low card levels) which require further investigation.**

A careful way to state the result is that the application now demonstrates substantially improved large-watchlist scalability after both the chunked backend seeding changes and the frontend first-paint/ progressive-activation optimisations, offset by a slight regression in ‘snappiness.’

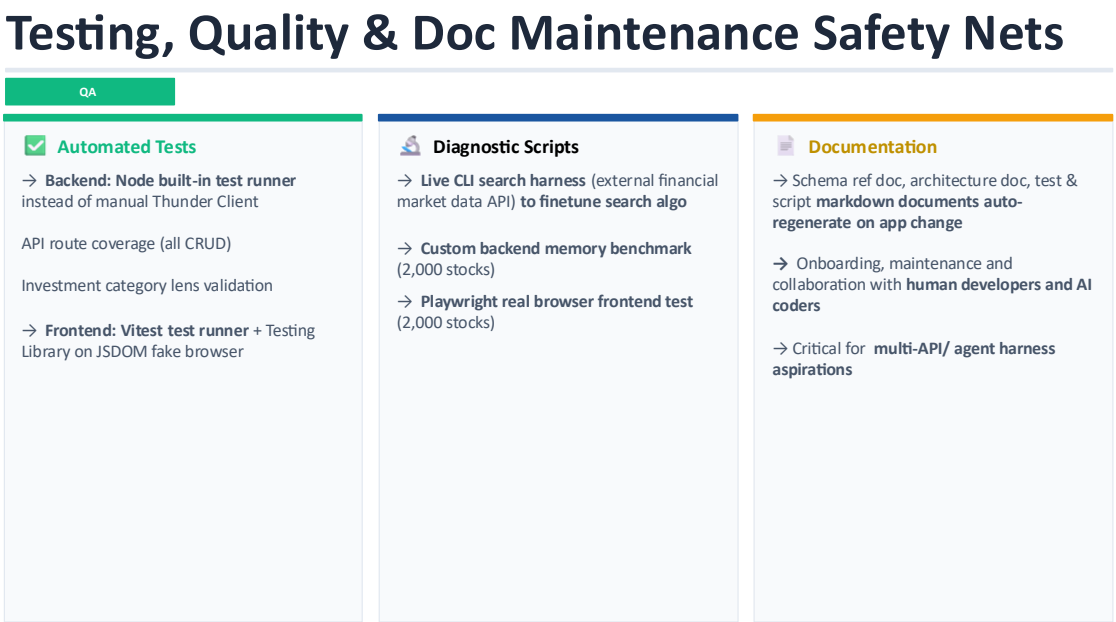
Project Planning

The original plan had a 3 day buffer period which I had allowed for additional items & explorations outside of the core feature set. Instead, this “free time” was quickly used up to make the existing core features operational.

Bug fixing (infinite React loops) and speed issues took literally days to resolve.

Testing, QA & Doc Maintenance

Testing Strategy Intro



Testing infrastructure is a major part of this project because the app is designed to keep growing over time. New APIs, new stock fields, more category rules, more performance work, and more data-manipulation logic all increase the risk of regressions. Because of that, this app uses several different kinds of testing.

In simple terms, this app uses:

- **frontend UI tests** to prove the visible React behaviour works
- **backend service and route tests** to prove the Express + MongoDB logic works
- **end-to-end tests** to prove realistic import/update/delete workflows work
- **performance harnesses** to prove the app still behaves acceptably at large watchlist sizes
- **manual scripts** to help human developers inspect live search behaviour and field visibility rules, as well as test performance at scale
- **documentation tooling** to keep the schema reference, architecture diagram and list of tests & scripts automatically sync with the source code

1. Frontend / UI automated tests

These tests mainly live under src/**/__tests__/ and are run with **Vitest** and **JSDOM**.

Shared search and page-level behaviour

src/contexts/__tests__/StockSearchContext.test.jsx
Tests the shared stock-search state used across the app. It verifies watchlist summary loading, stock add/open/remove flows, and shared search error handling.

src/components/__tests__/StockSearchResults.test.jsx
Tests the visible search-results UI. It checks that the app shows the right action for each result, especially SEE STOCK for existing watchlist stocks and ADD STOCK for new ones.

src/components/__tests__/StockSearchResults.layout.test.jsx
Tests the width/layout contract for the shared search-results component so it stays visually consistent across pages.

src/pages/__tests__/Home.search-results-layout.test.jsx
Checks that the Home page keeps the shared search-results width and alignment correct even though the page uses a different layout wrapper from Stocks.

Home page category cards and charts

src/components/__tests__/SectorChart.test.jsx
Tests the investment-category chart behaviour, including date ranges, preset movement, label behaviour, hover tooltip positioning, and shared scrollbar rules.

src/components/__tests__/SectorCardComponent.test.jsx
Tests the homepage investment-category card UI. It covers constituent opening, constituent toggling, layout behaviour, default trailing-range behaviour, and related card interactions.

src/services/__tests__/investmentCategoryCardsApi.test.js
Tests the frontend API wrapper used by the Home page category-card system. It checks request/response handling, normalization, and canonical initial range behaviour.

Stocks page orchestration

src/pages/__tests__/Stocks.test.jsx
This is one of the most important frontend tests in the app. It now covers:

- summary shells rendering before full dashboard bootstraps finish
- the bounded initial render window
- render-window growth as the user scrolls
- small queued dashboard bootstrap requests
- deferred legacy refresh after first visible real dashboard paint
- focused metrics mode and restoring the full stock-card list afterwards

src/pages/__tests__/StocksRemount.integration.test.jsx
A larger integration-style UI test around Stocks route remount behaviour and focused-card restoration flows.

Stock dashboard UI and metrics interactions

src/components/__tests__/SharePriceDashboard.test.jsx

This is the deepest frontend UI suite in the app. It covers:

- stock chart preset behaviour and horizontal scroll syncing
- lazy metrics loading
- focused metrics mode
- row-action menus
- hide/show and bold/unbold row behaviour
- override editing behaviour
- currency-row display rules
- section boundary rendering
- tooltip behaviour
- scrollbar behaviour
- several difficult React regression cases

src/components/__tests__/sharePriceChartScale.test.jsx
Tests stock-chart Y-axis scale logic and label formatting.

Frontend service / normalization tests

src/services/__tests__/watchlistDashboardApi.test.js
Tests the frontend watchlist dashboard API layer. It covers:

- payload normalization
- default bold-row behaviour
- currency rows
- derived-field read-only behaviour
- chunked dashboard bootstrap requests
- lazy metrics loading
- row-preference update contracts

2. Backend automated tests

These tests mainly live under tests/ and are run with Node's built-in test runner.

Backend service and business-rule tests

tests/stock-search-service.test.js
Tests the stock search service logic in isolation. It covers:

- ticker-vs-name query handling
- exchange suffix variants
- fallback word broadening
- duplicate merging
- ranking behaviour
- latest-price enrichment
- upstream error handling

tests/watchlist-dashboard-service.test.js
Tests backend shaping for the Stocks page dashboard bootstrap payload, including row-preference loading.

tests/stock-metrics-view-service.test.js
Tests the backend service that shapes the detailed metrics view, including visibility rules, default bold rows, derived-field behaviour, and saved preference precedence.

tests/investment-category-cards-service.test.js
Tests the backend logic for building homepage investment-category card data, including price condensation, re-indexing, averaging, and default range selection.

tests/import-range-behavior.test.js
Tests how import range caps, annual history behaviour, earnings-release-date fallback logic, and currency normalization work during stock import.

tests/roic-annual-fetch-options.test.js
Tests how annual-history fetch options are sent to the external ROIC API.

tests/default-bold-stock-rows.test.js
Tests the shared source of truth for default-bold stock rows.

tests/default-bold-stock-rows-vite.test.js
Tests that the browser-facing default-bold helper still works correctly through Vite/browser module loading.

HTTP route and integration tests

tests/frontend-api-routes.test.js
Tests the read-only backend HTTP routes for stock search and stock prices. It starts the real Express app, stubs the service layer, sends real HTTP requests, and checks validation, response shape, and upstream error translation.

tests/watchlist-routes.test.js
Tests the watchlist-related backend routes, including:

- lightweight summary route
- batched dashboard bootstrap route
- metrics-view route
- row-preference persistence
- override restrictions on derived fields

tests/investment-category-cards-routes.test.js
Tests the homepage category-card routes, including aggregation, default ranges, constituent toggles, and validation.

tests/lens-visibility.test.js
Integration test for the Lens system. It seeds lens data and verifies that each investment category resolves to the expected visible fields for both card and detail surfaces.

tests/missing-earnings-calls-import.test.js
Tests a real edge case where the external API returns a special “no earnings calls found” result.

Startup, schema and migration support tests

tests/server-startup.test.js
Tests that the backend can still start and expose read-only stock search even if MongoDB is unavailable at boot.

tests/watchlist-stock-model-load.test.js
Simple model-load smoke test for the main stock model.

tests/schema-reference-generator.test.js
Sanity test for generated schema documentation. It checks that important schema-reference sections still appear and fails fast if required sections would be empty.

tests/inspect-lens-fields.test.js
Smoke test for the CLI lens inspection script.

tests/investment-category-migration.test.js
Tests investment-category migration behaviour.

3. End-to-end backend workflow tests

These tests use more of the real system at once.

tests/e2e-stubbed-import-crud.test.js
Deterministic end-to-end backend test with stubbed external financial API data, but real Express routes, normalization, and MongoDB persistence. It covers:

- import
- read/list
- update
- annual overrides
- forecast overrides
- top-level metric overrides
- refresh
- delete

tests/e2e-live-import-crud.test.js
Live end-to-end backend confidence test against the real external financial API and real MongoDB. It exercises the same general workflow as the stubbed E2E test, but with live upstream data.

A beginner way to think about the difference is:

- **stubbed E2E** = safer, repeatable, controlled
- **live E2E** = closer to production behaviour, but dependent on real outside data

4. Performance and scalability tests

These are heavier than normal correctness tests.

Browser benchmark

tests/performance/browserBenchmark.js
This is the real Playwright browser benchmark used to measure how the Stocks page behaves under large watchlists.

It measures things such as:

- first visible shell render
- first visible real dashboard render
- first usable interaction
- progressive dashboard activation during scroll
- browser heap growth
- whether background refresh starts too early

This benchmark is especially important because it tests **real user-facing performance**, not just correctness.

Backend performance harness

tests/performance/backendHarness.js
Custom backend performance harness that measures:

- route latency
- payload size
- heap memory growth
- RSS memory growth
- repeated-run drift

It focuses on the routes most important to scale, especially watchlist and homepage category-card routes.

Performance dataset and helpers

tests/performance/largeWatchlistDataset.js
Creates deterministic large watchlist datasets for performance testing.

tests/chunked-large-watchlist-seeding.test.js
Tests that the chunked seeding logic works correctly and inserts the expected records.

tests/performance/baselines/*.json
Checked-in baseline files used for relative benchmark comparisons.

These performance tools differ from the rest of the test suite because they are not mainly checking correctness. They are checking whether the app remains usable and bounded when the watchlist becomes very large.

5. Manual diagnostic scripts

These are useful when a developer wants to inspect behaviour directly.

Live search CLI

scripts/live-stock-search-cli.js
Manual live search harness for the backend search logic.

What it does:

- loads .env
- requires a real external API key
- prompts for a ticker or company name
- calls the search service directly
- prints a diagnostic results table

Why it is useful:

- quick manual inspection of live search behaviour
- helpful for unusual or messy real-world search inputs
- shows ranking sources and name-source diagnostics
- surfaces live upstream quirks that are hard to simulate perfectly

Lens inspection CLI

scripts/inspect-lens-fields.js
Developer-facing CLI for inspecting what fields a category or stock can see under the Lens system.

Why it is useful:

- gives a fast backend-only check of visibility rules
- helps confirm which fields belong in card vs detail view
- works without opening the frontend

Branch sync helper

scripts/check-frontend-branch-sync.js
Developer utility script for checking frontend branch sync status.

Stock backfill helper

scripts/backfill-watchlist-stocks.js
Utility script related to watchlist stock backfill / repair work.

6. Documentation tooling

Schema reference generator

`scripts/generate-schema-reference.js`

Generates the schema reference from the field catalog and writes it to `docs/schema-reference.md`.

This is useful because it keeps documentation aligned with the actual schema source of truth and makes the stock document shape easier to browse. It summarizes field paths, source types, endpoint mappings, and category visibility.

Beginner architecture doc generator

`scripts/architecture-doc-source.js`

Structured source-of-truth module for the beginner architecture doc. When a major architecture change happens, this is the file developers edit.

`scripts/generate-beginner-architecture-doc.js` generates `docs/beginner-architecture-diagram.md` from that structured source module.

`npm run docs:architecture` is the local regeneration command for the architecture doc.

`npm run test:docs:architecture` runs the focused architecture-doc generator test.

`npm run test:docs` now runs both generated-doc checks: the schema reference generator test and the beginner architecture generator test.

This tooling is useful because it keeps the architecture diagram aligned with the app's current layers, flows, routes, and key file map while still giving beginners a readable high-level document.

Test and script overview doc generator

`scripts/test-and-script-overview-source.js`

Structured source-of-truth module for this document. If tests, scripts, or generated-doc workflows change in a meaningful way, this is the file to update.

`scripts/generate-test-and-script-overview.js` generates `docs/test-and-script-overview.md` from that structured source module.

`npm run docs:test-overview` is the local regeneration command for this overview doc.

`npm run test:docs:test-overview` runs the focused stale-output and anchor test for this overview doc.

This workflow is useful because the overview itself changes whenever the repo gains or removes important tests, scripts, or generated-doc tooling. The generator keeps the checked-in markdown deterministic, while the test makes sure the committed file stays aligned with **the** source module.

Implementation

What are my considerations for deploying the software?

Post Capstone, it is my intention to deploy the application using a containerised architecture on Amazon Linux, leveraging Docker to package the application and its dependencies into a reproducible image that can be run consistently across environments.

The deployment will involve provisioning an Amazon Linux host, installing and configuring Docker, building and running the application container with appropriate port mappings, environment variables and persistent storage where required.

To ensure the application remains up to date with minimal manual intervention, Watchtower will be configured to monitor the running container and automatically pull and deploy updated image versions from the container registry. This enables near-continuous deployment workflow, whereby each commit to GitHub triggers an image update, with changes propagated to the live application within a maximum of 5 minutes.

End-to-end solution

How well did the software meet its objectives? What are next steps?

The software met its objectives as a working prototype toward the desired end state of **"freeing up time for traditional equity investors to think more clearly and make more confident portfolio decisions."**

The prototype app successfully brings together several important parts of an investment research workflow. It allows users to search for stocks, add them to a watchlist, save stock data in MongoDB, view financial metrics, compare stock dashboards, and organise companies into investment categories. This means the app is starting to move beyond being a simple stock lookup tool. We have the raw beginnings of a structured research assistant that helps investors reduce manual work and keep their thinking organised.

A major strength of the software is that it separates live external API data from saved database data. This helps the app avoid unnecessary API calls and makes the user experience faster once stocks have already been imported. The use of MongoDB also means users can keep their watchlist, preferences, overrides, and category choices instead of starting again each session.

However, the app still has many areas that need to be improved to turn it into a minimal viable product to support day-to-day self-use, let alone external users or a contractual tender with a foundational customer.

One important next step is to fine-tune the performance versus resource-use trade-off. As the watchlist grows, the app needs to stay fast without making too many API calls, loading too much data at once, or using too much browser memory. There is already work in the app to improve this, such as loading dashboard data in smaller batches and lazily loading detailed metrics only when needed. But further testing and optimisation is required to manage speed degradation.

Another area for improvement is the refresh strategy. At the moment, the app can refresh stock data manually or when saved data is structurally outdated, but it could be improved by adding a clearer time-based refresh rule. For example, the app could check whether saved fundamentals are older than six months and automatically mark them as needing an update. This would make the system smarter and more reliable for long-term users.

The next steps would be to:

- add a clearer automatic refresh policy for old financial fundamentals
- continue improving performance for large watchlists
- reduce unnecessary backend and external API calls
- improve loading states and error handling for users
- expand testing around scalability, refresh behaviour, and real-world usage
- refine the user interface so investors can compare companies even faster

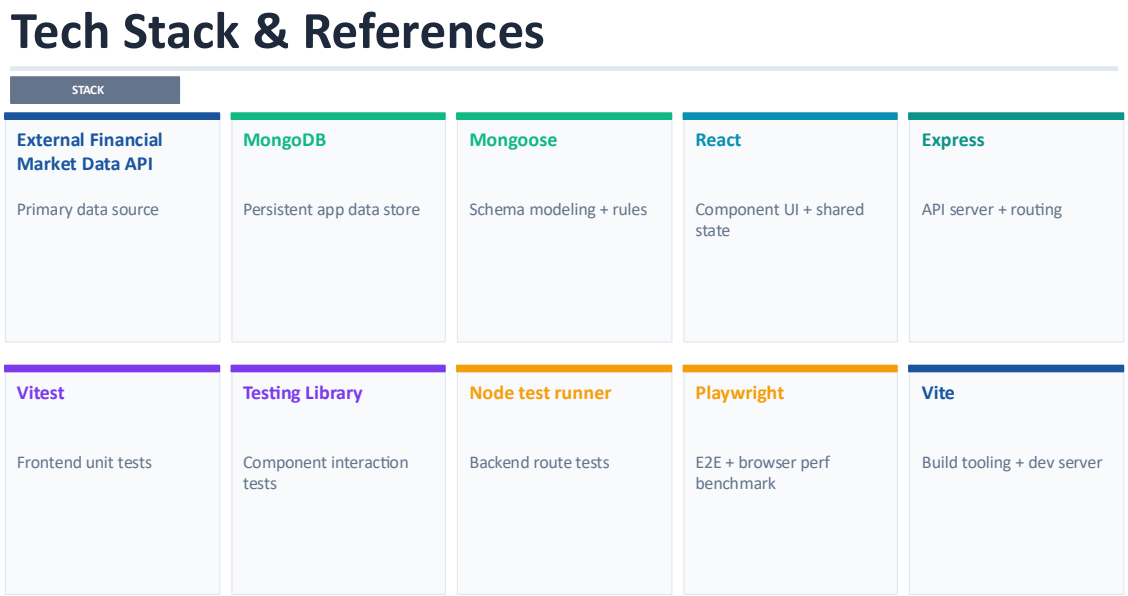
After this, we can move on to further feature sets shown in the **Out of Scope section**, being immediate usability improvements such as Home Page horizontal scrolling, investment category filtering, drag-and-drop row and stock-card ordering, and better currency-aware chart labels; then medium-term product features such as user accounts, custom investment categories and lenses, richer financial metrics, multiple market-data APIs, news and broker-report integration, AI-assisted override entry, and eventually portfolio-weight tracking so users can quickly analyse “aggregate fundamentals” of their portfolio.

References

Code location

MAIN
- backend-branch
- frontend branch

Key resources



Backend & Data Layer

Express – server framework used to define the backend API, routes, controllers, and JSON responses.

Axios – promise-based HTTP client used by the backend to call the external ROIC financial data API.

Mongoose – ODM library used to define schemas and interact with MongoDB collections as application models.

MongoDB – the database used to store the app's persistent data such as watchlist stocks, row preferences, constituent preferences, and cached price history.

Dotenv – environment variable loader used for sensitive configuration such as API keys, ports, and database connection strings.

External Financial Market Data API – third-party API used for stock search, company profile data, fundamentals, earnings-call-related data, and share-price history.

Frontend

React – core frontend library used to build the component-based UI and shared client-side application logic.

React DOM – React renderer used to mount the React application into the browser DOM.

Vite – frontend dev server and build tool used for fast local development and production bundling.

React Router DOM - client-side routing library used to switch between the Home and Stocks pages without full page reloads.

Material UI (MUI) – React UI component library used for cards, dialogs, buttons, layout primitives, alerts, typography, and general interface structure.

Emotion – styling engine used underneath MUI for component styling and theme-aware CSS-in-JS behaviour.

Development & Testing Tools

Vitest – Vite-native test runner used for frontend unit and component tests.

Playwright – browser automation framework used in this project for the real-browser Stocks page benchmark and large-watchlist performance testing.

Testing Library (@testing-library/react) – UI testing utility used to render React components and assert visible behaviour in tests.

Testing Library User Event (@testing-library/user-event) – helper library used to simulate realistic user interactions such as clicks, typing, and keyboard input.

JSDOM – browser-like DOM environment used by frontend tests so React UI can be tested inside Node.js.

Node.js built-in test runner (node:test) – used for backend tests and route/service integration tests.

Nodemon – development tool that automatically restarts the backend server when files change.

ESLint – static analysis and linting tool used to enforce code quality and consistency across the repo.

Key files list

This can be found in docs\beginner-architecture-diagram.md within the app.

Presentation layer

Navigation + Search UI navbar

The key files for the Navigation + Search UI part of the presentation layer are:

src/components/NavBar.jsx (line 1)

This is the main navigation bar. It renders the menu, app title, and the search input/form the user types into.

src/components/StockSearchResults.jsx (line 1)

This is the shared visible search-results panel. It shows loading/error states plus the ADD STOCK / SEE STOCK actions.

src/components/stockSearchResultsLayout.js (line 1)

This holds the shared layout contract for the search-results shell so Home and Stocks render it consistently.

The key supporting files just behind that UI are:

src/contexts/StockSearchContext.jsx (line 98)

This is the shared state and behavior behind the navbar search and search results.

src/hooks/useStockSearch.js (line 1)

Small hook that gives UI components access to the shared search state.

src/App.jsx (line 1)

Important because it wires the NavBar and the StockSearchProvider together at the app level.

Home page

For the Home page part of the presentation layer, the key files are:

src/pages/Home.jsx (line 1)

This is the main Home page container. It loads the category cards and renders the page layout.

src/components/SectorCardComponent.jsx (line 1)

This is the main UI card shown on the Home page for each investment category.

src/components/SectorChart.jsx (line 1)

This renders the chart inside each category card.

src/components/StockSearchResults.jsx (line 1)

Home also shows the shared search results UI near the top of the page.

Supporting-but-not-pure-presentation files behind Home are:

src/services/investmentCategoryCardsApi.js (line 1)

Frontend API adapter Home uses to fetch and update category-card data.

src/components/stockSearchResultsLayout.js (line 1)

Shared layout helper used by the search-results area shown on Home.

Stocks Page

For the Stocks Page part of the presentation layer, the key files are:

src/pages/Stocks.jsx (line 1)

This is the main Stocks page container. It loads the watchlist dashboard cards, handles page-level states, and renders the overall page layout.

src/components/SharePriceDashboard.jsx (line 1)

This is the main stock card UI for each watchlist item. It displays the share price chart, stock details, metrics view, and row-level interactions.

src/components/StockSearchResults.jsx (line 1)

This is also part of the Stocks page presentation because the page shows the shared search-results panel above the watchlist cards.

Supporting files behind the Stocks page are:

src/services/watchlistDashboardApi.js (line 1)

Main frontend API entry point for stock dashboard data.

src/contexts/StockSearchContext.jsx (line 98)

Shared state used by the Stocks page for watchlist summary, add/open/remove flows, and search integration.

src/hooks/useStockSearch.js (line 1)

Hook used by Stocks page components to access that shared state.

Frontend App layer (React)

This is the React app, the JavaScript code running in the browser. This layer sits between the Presentation layer, and what lives on the server i.e. the backend.

a. Shared Stock Search State

Key files:

[`src/contexts/StockSearchContext.jsx`](../src/contexts/StockSearchContext.jsx)
Shared search and watchlist state used across pages.

Supporting files:

[`src/hooks/useStockSearch.js`](../src/hooks/useStockSearch.js)
Hook that lets components consume the shared search state.

[`src/App.jsx`](../src/App.jsx)
Wraps the app in `StockSearchProvider`.

b. Frontend API Services

Key files:

[`src/services/investmentCategoryCardsApi.js`](../src/services/investmentCategoryCardsApi.js)
Frontend API adapter for Home page category cards.

[`src/services/watchlistDashboardApi.js`](../src/services/watchlistDashboardApi.js)
Public frontend API entry point for watchlist dashboard reads and mutations.

Supporting files:

[`src/services/watchlistDashboardApi.reads.js`](../src/services/watchlistDashboardApi.reads.js)
Dashboard data loading helpers.

[`src/services/watchlistDashboardApi.mutations.js`](../src/services/watchlistDashboardApi.mutations.js)
Dashboard update/mutation helpers.

[`src/services/watchlistDashboardApi.normalizers.js`](../src/services/watchlistDashboardApi.normalizers.js)
Browser-side payload normalization.

The Bridge: HTTP + JSON over /api/*

Key files:

[`src/services/investmentCategoryCardsApi.js`](../src/services/investmentCategoryCardsApi.js)
Sends frontend requests into `/api/homepage/...`.

[`src/contexts/StockSearchContext.jsx`](../src/contexts/StockSearchContext.jsx)
Sends shared search/watchlist requests into `/api/stocks/...` and `/api/watchlist/...`.

[`src/services/watchlistDashboardApi.reads.js`](../src/services/watchlistDashboardApi.reads.js)
Sends dashboard read requests into `/api/watchlist/...`.

[`src/services/watchlistDashboardApi.mutations.js`](../src/services/watchlistDashboardApi.mutations.js)
Sends dashboard mutation requests into `/api/watchlist/...`.

[`server.js`](../server.js)
Mounts the backend route groups under `/api`.

Supporting files:

[`routes/stockLookupRoutes.js`](../routes/stockLookupRoutes.js)
Read-only stock lookup endpoints.

[`routes/watchlistRoutes.js`](../routes/watchlistRoutes.js)
Watchlist and dashboard endpoints.

[`routes/investmentCategoryCardsRoutes.js`](../routes/investmentCategoryCardsRoutes.js)
Homepage category-card endpoints.

The Backend API Layer

a. Stock Lookup API

Key files:

[`routes/stockLookupRoutes.js`](../routes/stockLookupRoutes.js)
Route definitions for live search and price lookup.

[`controllers/stockLookupController.js`](../controllers/stockLookupController.js)
Request validation and JSON response handling for stock lookup.

Supporting files:

[`server.js`](../server.js)
Mounts the stock lookup API under `/api`.

b) Watchlist API

Key files:

[`routes/watchlistRoutes.js`](../routes/watchlistRoutes.js)
Route definitions for summary, dashboards, import, CRUD, metrics, and refresh.

[`controllers/watchlistController.js`](../controllers/watchlistController.js)
Core watchlist CRUD, summary, and dashboard bootstrap handlers.

[`controllers/importController.js`](../controllers/importController.js)
Stock import endpoint.

`controllers/stockMetricsViewController.js`](../controllers/stockMetricsViewController.js)
Metrics-view and row-preference endpoints.

[`controllers/refreshController.js`](../controllers/refreshController.js)
Stock refresh endpoint.

[`controllers/overrideController.js`](../controllers/overrideController.js)
Override update endpoints.

Supporting file:
[`server.js`](../server.js)
Mounts the watchlist API under `/api/watchlist`.

c. Homepage Category API

Key files:

[`routes/investmentCategoryCardsRoutes.js`](../routes/investmentCategoryCardsRoutes.js)
Route definitions for homepage category-card queries and updates.

[`controllers/investmentCategoryCardsController.js`](../controllers/investmentCategoryCards Controller.js)
Request validation and JSON responses for homepage category cards.

[`server.js`](../server.js)
Mounts the homepage category API under `/api/homepage/investment-category-cards`.

The Backend Business/Data Access Layer

a. Stock Search / external financial market data API Integration Service

Key files:

[`services/stockSearchService.js`](../services/stockSearchService.js)
Search classification, ranking, deduplication, and result shaping.

[`services/roicService.js`](../services/roicService.js)
Low-level external financial market data API client used to fetch external market/company data.

Supporting file:

[`services/stockSearchApi.js`](../services/stockSearchApi.js)

Backend helper around read-only stock lookup flows.

b) Watchlist & Dashboard Service

Builds stock dashboard data from saved watchlist data + price data.

Key files:

[`services/watchlistDashboardService.js`](../services/watchlistDashboardService.js)

Summary and dashboard bootstrap logic for the Stocks page.

[`services/stockMetricsViewService.js`](../services/stockMetricsViewService.js)

Metrics-view shaping and row-preference support.

[`services/watchlistStockRefreshService.js`](../services/watchlistStockRefreshService.js)

Refreshes external financial market API stock data while preserving user data.

Supporting files:

[`services/normalizationService.js`](../services/normalizationService.js)

Converts imported external financial market API data into the app's stock document shape.

[`services/stockDataVersionService.js`](../services/stockDataVersionService.js)

Decides whether stored stock documents need refresh or upgrade.

c. Homepage Category Cards Service

Key file:

[`services/investmentCategoryCardsService.js`](../services/investmentCategoryCardsService.js)

Builds homepage category-card payloads and updates constituent state.

Supporting file:

[`services/lensService.js`](../services/lensService.js)

Validates and prepares investment-category/lens configuration used by homepage cards.

d. MongoDB Models / Persistence

Key files:

[`models/WatchlistStock.js`](../models/WatchlistStock.js)

Main watchlist stock document.

[`models/StockMetricsRowPreference.js`](../models/StockMetricsRowPreference.js)

Persisted row-level preferences for stock cards.

[`models/InvestmentCategoryConstituentPreference.js`](../models/InvestmentCategoryConstituentPreference.js)

Persisted constituent toggles for homepage cards.

[`models/StockPriceHistoryCache.js`](../models/StockPriceHistoryCache.js)

Cached price history used by homepage category cards.

[`models/Lens.js`](../models/Lens.js)

Lens/category configuration stored in MongoDB.

Supporting file:

[`config/db.js`](../config/db.js)

MongoDB connection setup used by the Express server.

Database schema

The most important design idea in my DB is that I kept financial data separate from UI preferences and display rules. This has the following benefits:

- the stock data stays clean
- I can customise the UI without corrupting the underlying research data
- category display logic can evolve separately from the stored stock records

1. watchlist

This is the main collection in my app. It stores:

- one document per stock in my watchlist
- the stock's ticker and company name
- its investment category
- annual financial history
- forecast placeholders
- growth forecasts
- analyst revision fields
- metadata about when the stock was imported or refreshed

This is the collection that holds the actual "stock research record".

A good beginner way to describe it is:

"This is my master stock table. Each document is one stock, with its identity, imported financial data, manual overrides, and forecast fields."

One important technical detail is that many financial fields use the same override-capable structure:

- roicValue
- userValue
- effectiveValue
- sourceOfTruth

That means my DB does not just store a number. It often stores:

- the API value
- the user override
- the final value the app should use

2. stock_price_history_cache

This collection stores a cached monthly price history for each stock. It stores:

- one document per ticker
- a compact list of monthly closing prices
- the earliest month in the cache
- the latest month in the cache
- when the cache was last synced

I use this mainly so the app does not have to keep re-fetching and re-processing full price history every time the Home page category cards need to build their price index charts.

A simple way to explain it is:

“This is my fast price-history cache. It stores monthly chart data for each stock so the homepage category charts can load faster.”

3. stock_metrics_row_preferences

This collection stores row-level display preferences for the stock dashboard. It stores:

- one record for one stock row
- whether that row is hidden or shown
- whether that row is bold or not bold

So this collection is not financial data. It is presentation preference data.

A simple explanation is:

“This stores how I want individual metric rows to look on each stock card, like hidden/shown and bold/unbold.”

The key idea is:

- watchlist stores the stock data itself
- stock_metrics_row_preferences stores how I want that data displayed

4. lenses

This collection stores the display blueprint for each investment category. It stores:

- which fields should appear for a category
- what label each field should use
- what order fields should appear in
- whether a field belongs on the card surface or the detailed surface

So this is like a configuration collection for category-specific views.

A simple explanation is:

"This tells the app which metrics each investment category should see, in what order, and in which part of the UI."

For example, a Lender lens and a Profitable Hi Growth lens do not need to show the same fields. This collection tells the backend what each category's view should contain.

5. investment_category_constituent_preferences

This collection stores whether a stock is enabled or disabled inside a category card. It stores:

- one record for one stock inside one investment category
- whether that stock should count as an active constituent for that category card

This is separate from the stock's actual investmentCategory in watchlist.

That is important because:

- a stock can still belong to a category
- but I may temporarily disable it from the category index calculation

A simple explanation is:

"This stores whether a stock should be included or excluded from a category card's constituent list and price index."

External Systems

a. External financial market data API

Key file:

[`services/roicService.js`](../services/roicService.js)

The main backend boundary to the external financial market data API.

Supporting files:

[`services/stockSearchService.js`](../services/stockSearchService.js)

Uses external financial market API data to build search results.

[`services/watchlistStockRefreshService.js`](../services/watchlistStockRefreshService.js)

Uses external financial market API backed datasets during import/refresh workflows.

[`services/investmentCategoryCardsService.js`](../services/investmentCategoryCardsService.js)

Uses external financial market API backed price data for homepage cards.

b. MongoDB Database

Key files:

[`config/db.js`](../config/db.js)

Opens and closes the database connection.

[`server.js`](../server.js)

Starts the app and connects to MongoDB during backend startup.

Supporting files:

These files are my app's "database blueprints." They tell MongoDB what kind of data to store and how that data is organized.

[`models/WatchlistStock.js`](../models/WatchlistStock.js)

This is the main blueprint for a stock in our watchlist. It stores the stock's ticker, company name, investment category, currencies, yearly financial data, forecast data and metadata about when the stock was imported or refreshed.

[`models/StockMetricsRowPreference.js`](../models/StockMetricsRowPreference.js)

This stores display preferences for one row on one stock card. For example, whether a row should be shown/ hidden or bold/ unbold.

[`models/InvestmentCategoryConstituentPreference.js`](../models/InvestmentCategoryConstituentPreference.js)

This stores whether a stock is enabled or disabled inside a Home page investment category card. It does not change the stock itself. It only changes whether that stock is included in that category-card view.

[`models/StockPriceHistoryCache.js`](../models/StockPriceHistoryCache.js)

This stores cached monthly price history for a stock. The app uses it so it does not have to keep asking the external financial data API for the same price history over and over.

[`models/Lens.js`](../models/Lens.js)

This stores the rules for what fields a category should show, what labels to use, what order to show them in, and whether they belong on the card or in detailed view.

Mongoose schema structure

Category	Collection / Schema & description	What it stores	Main fields / structure
Core business data	**watchlist via WatchlistStock** This is the central document in the app. It holds both imported financial data and user-editable forecasting fields.	The main stock record for each company in the watchlist	tickerSymbol, companyName, investmentCategory, priceCurrency, reportingCurrency, sourceMeta, annualData[], forecastData, growthForecasts, analystRevisions
Historical yearly data	**annualData[] subdocument inside WatchlistStock** This keeps each stock's historical financials organised year by year.	One row per fiscal year for a stock	fiscalYear, fiscalYearEndDate, reportingCurrency, earningsReleaseDate, plus grouped sections like base, balanceSheet, incomeStatement, valuationMultiples, epsAndDividends
Forward-looking estimates	**forecastData subdocument inside WatchlistStock** This supports the app's future assumptions and valuation work.	Manual and derived forecast values for FY+1 FY+2 and FY+3	fy1, fy2, fy3, each containing fields like ebit, pe, eps, dps, marketCap, enterpriseValue
Growth assumptions	**growthForecasts subdocument inside WatchlistStock** This stores high-level growth expectations used in analysis.	User-entered growth assumptions	revenueCagr3y, revenueCagr5y, epsCagr3y, epsCagr5y
Analyst trend inputs	**analystRevisions subdocument inside WatchlistStock** This captures short-term analyst estimate changes for the stock.	Manual analyst revision tracking	revenueFy1Last1m, ebitFy2Last3m, epsFy3Last1m and related revision fields
Shared metric field shape	**metricFieldSchema subdocument used across WatchlistStock** This is how the app supports imported values, user overrides, and derived values in one consistent format.	The reusable structure for most stored metrics	roicValue, userValue, effectiveValue, sourceOfTruth, baseSourceOfTruth, lastOverriddenAt
UI row preferences	**stock_metrics_row_preferences via StockMetricsRowPreference**	Per-stock display preferences for card and detail rows	tickerSymbol, rowKey, isEnabled, isBold

	This keeps presentation choices separate from financial data. Hiding or bolding a row does not change the stock's actual metrics.		
Category card preferences	**investment_category_constituent_preferences via InvestmentCategoryConstituentPreference** This lets the user include or exclude stocks from category summaries without changing the stock's saved category.	Whether a stock is included in a category card view	investmentCategory, tickerSymbol, isEnabled
Lens / display configuration	**lenses via Lens** This acts like a display blueprint. It tells the app what metrics belong on stock cards versus detailed views for each category.	Defines which fields each investment category should show	key, name, normalizedName, isActive, fieldConfigs[] where each config has fieldPath label section order and surface
Price cache	**stock_price_history_cache via StockPriceHistoryCache** This improves performance by avoiding repeated processing of full historical price series.	Cached monthly price history for faster category-card charts	tickerSymbol, pricePoints[], earliestMonth, latestMonth, lastSyncedAt

Concept	Meaning in this app
Document	A single MongoDB record such as one stock in watchlist
Collection	A group of related documents such as watchlist or lenses
Subdocument	A nested structure inside a larger document such as annualData[] or forecastData
Override model	The app stores original imported values user-entered values and the final effective value together
Separation of concerns	Financial data lives in watchlist while UI preferences and display rules live in separate collections